

Comparative Analysis of Sequential Models for Mobile Network Traffic Forecasting

Emmanuel NSABAGASANI — African Leadership University Formative Assignment 1, September 2026

Abstract

Short-horizon forecasts of cellular traffic underpin dynamic resource allocation and base-station energy saving, yet the radio access network still accounts for 73–87% of mobile-operator energy consumption. This study compares three structurally different sequential models — a seasonal ARIMA, an LSTM, and a temporal convolutional network (TCN) — for one-step-ahead (10-minute) forecasting of Internet activity on the Telecom Italia Milan grid, and asks how their performance varies across geographical areas with different traffic characteristics. The 19.4 GB, 320-million-row source dataset was reduced to a 341 MB memory-mapped `float32` store by a streaming pipeline that aggregates during parsing; peak resident memory fell 2.7× and the in-memory payload 27×, never exceeding 241 MB against a 1 GB budget. Models were tuned on the highest-traffic area through a documented sequential search and applied unchanged to three areas, evaluated on 16–22 December 2013.

The central finding concerns the benchmark. All models beat the seasonal-naive reference by a wide margin, giving MASE values far below 1, but seasonal naive is weak for this series: lag-1 autocorrelation is 0.987 against 0.878 at lag 144. Measured against persistence, SARIMA is 7.6–16.4% *worse* on every area. The TCN improved on persistence by 14.8%, 15.4% and 15.5% — a spread of 0.007 across areas that differ fivefold in level and invert in weekend behaviour — while the LSTM achieved a similar mean (0.905 versus 0.848) with a spread 25 times larger, ranging from a 17.5% gain to no gain at all. The TCN also trained faster than the LSTM with a third of its parameters. A controlled experiment with matched epochs found **no** measurable effect of receptive field between 61 and 509 steps: the strong daily seasonality identified in the exploratory analysis is real but largely redundant at a one-step horizon, decisive for a linear model that must encode it explicitly and nearly irrelevant to a network conditioned on recent history. Errors concentrate almost entirely at the daily peak, and the worst window in the busiest area was the pre-Christmas shopping Saturday, a traffic level absent from the training period — a failure that is calendar-driven and therefore invisible to any univariate model.

1. Introduction

Mobile network operators allocate radio and transport capacity against demand that varies by an order of magnitude within a single day and by a further order of magnitude between neighbouring city blocks. Decisions that depend on knowing that demand a few minutes in advance — dynamic spectrum and bandwidth allocation, load balancing between cells, admission control, and switching lightly loaded cells into energy-saving states — are all short-horizon forecasting problems. Getting them wrong is costly in both directions: under-provisioning degrades service exactly when demand is highest, while over-provisioning wastes energy in a sector where radio access networks account for the majority of an operator's electricity consumption.

This study investigates that problem empirically on the Telecom Italia Milan dataset, which records telecommunications activity over a 100×100 grid of 10,000 geographical areas at 10-minute resolution for two months. The scale matters methodologically as well as practically: at 20.6 GB of raw text and roughly 340 million rows, the dataset does not fit in the memory of the machine used here, so how the data is handled is part of the research problem rather than a preliminary.

Research question. *How do different sequential models compare for one-step-ahead mobile network traffic forecasting, and how does their performance vary across geographical areas with different traffic characteristics?*

The second half of that question is the more interesting one. It is straightforward to report that one model achieves the lowest error on one cell; it is considerably more useful to know whether that ranking survives a change of location, and if not, what property of the traffic explains the change. The study therefore evaluates every model on three areas rather than one, and deliberately connects the results back to the temporal structure identified during exploratory analysis.

Objectives.

1. Design and quantify a data-handling strategy that makes a 20.6 GB dataset tractable within roughly 1 GB of working memory, and state its trade-offs.
2. Characterise the temporal and spatial structure of the Internet traffic, and use that characterisation — not convention — to determine the input representation and model configurations.
3. Implement three structurally different sequential models, tune them through a documented sequence of experiments, and evaluate them on a fixed held-out week.
4. Establish whether the additional complexity of the neural models is justified against strong naive baselines, and identify where and why the models fail.

Contributions. The main empirical findings are stated in Section 7. The methodological emphasis of the work is that the comparison is made honest: identical splits, identical scaling, a single evaluation harness, reference baselines reported throughout, and errors measured in original activity units.

2. Related Work

2.1 The dataset and the work built on it

The Telecom Italia Big Data Challenge release described by Barlacchi *et al.* [1] has become the de facto public benchmark for cellular traffic prediction. One detail from that paper governs the interpretation of every result below: the released quantity is an *activity measure*, obtained by attributing each Call Detail Record to a radio base station and then apportioning it to grid squares in proportion to the intersected area of the station's coverage map [1]. The series forecast here are therefore spatially smoothed counts of record-generating events, not byte volumes, which is why errors are reported in dimensionless activity units and compared only between models on the same area. The 100×100 grid of roughly 235 m squares and the 1 November 2013 – 1 January 2014 window are stated most explicitly in the downstream literature [2], [3]. The breadth of deep-learning application to mobile networking is surveyed by Zhang, Patras and Haddadi [9]; this study narrows immediately to short-horizon forecasting.

2.2 Classical statistical forecasting of network traffic

Seasonal ARIMA has been the standard linear approach in this domain since Shu *et al.* [10], whose contribution was an expression for seasonal ARIMA with *two* periodicities — directly relevant here, since at 10-minute resolution the daily period is 144 intervals and the weekly period 1,008. This study differs seasonally at lag 144 only. The weekly cycle is real (Section 4.3 finds a spectral peak near one week) but is not modelled, because a second seasonal difference at lag 1,008 would consume a further week of a 62-day series and add parameters the data cannot support.

The differencing order is chosen by diagnostic rather than convention, using the Augmented Dickey–Fuller test [19] and the KPSS test [20] together. Running both matters because their hypotheses are reversed — ADF nulls a unit root, KPSS nulls stationarity — so agreement is far stronger evidence than either alone. One caveat is stated honestly in Section 4.4: with thousands of observations per series the ADF test is very powerful and rejects almost everything, so the autocorrelation structure carries more weight than the p-value. Hyndman and Athanasopoulos [11] supply the seasonal-naive definition and the standard argument for a variance-stabilising transform.

This positioning is deliberate. Much of the deep-learning literature on this dataset compares against a *non-seasonal* ARIMA — Zhang and Patras [2], for instance, tune $p = 3$, $d = 1$, $q = 2$ with no seasonal term — on data whose dominant structure is a 24-hour cycle. A linear model denied the ability to represent that cycle is not a fair adversary. The SARIMA baseline used here is seasonally differenced and fitted on a log scale, which is the report's main claim to a like-for-like comparison.

2.3 Deep sequential models and what they have been shown to add

The architectural arc runs from the LSTM [12], introduced to solve decaying error backflow in recurrent networks, through ConvLSTM [15], which preserves spatial structure that a fully connected LSTM discards, to its application to cellular traffic in STCNet [4]. Purely convolutional spatial treatments [3], graph-based models [6], and early architecture comparisons on this same dataset [5] complete the picture.

What these papers actually claim requires care, and stating it precisely is what motivates the present study. Zhang and Patras [2] report "up to 61% smaller prediction errors", but the comparison is **long-horizon** — up to 60 steps, ten hours ahead — and the 61% figure is against Holt-Winters exponential smoothing; the corresponding figure against ARIMA is 35% [2]. Their paper also shows the deep models' margin *growing with horizon*, with the classical methods much closer at short horizons. The "4%–13%" improvement quoted for STCNet [4] is the marginal contribution of its transfer-learning step, not the total gain over a classical baseline. Zhang *et al.* [3] present their improvements graphically and make no quotable percentage claim.

Two further observations shaped this study's scope. First, Zhang *et al.* [3] explicitly aggregated the 10-minute data **up to hourly**, on the grounds that a large proportion of 10-minute cell values are zero and that "resource planning in 10 minutes level is a non-trivial task" [3]. This report does the thing that paper declined to do, which is part of why the near-zero night-time values and the choice of a $\log_{1p}$ transform receive as much attention as they do in Section 5.2. Second, Trinh *et al.* [7] are among the few to evaluate one-step-ahead performance separately from long-term performance, and to study how the length of the observed input window affects error — the same design question addressed by the lookback experiments in Section 6.1.

The gap this study occupies follows directly: the one-step-ahead, native 10-minute, single-cell problem is comparatively under-examined, and it is precisely the regime in which classical methods should be most competitive.

2.4 Convolutional versus recurrent sequence modelling

Dilated causal convolutions were popularised by WaveNet [14], where exponentially growing dilation buys a receptive field spanning thousands of samples at linear parameter cost while enforcing causality architecturally rather than through data handling. Bai, Kolter and Koltun [13] generalised this into the TCN and reported that "a simple convolutional architecture outperforms canonical recurrent networks such as LSTMs across a diverse range of tasks and datasets, while demonstrating longer effective memory", arguing that the long memory usually attributed to recurrent networks "is largely absent in practice" [13].

Two qualifications belong with that claim. Their evaluation suite comprises synthetic stress tests, polyphonic music and language modelling — none of them a strongly seasonal real-valued forecasting problem — and the work is an arXiv technical report rather than a peer-reviewed conference paper. Testing whether its conclusion transfers to a seasonal telecom series at a one-step horizon is therefore an open question, and is the specific comparison this study makes, with the LSTM [12] as the matched recurrent comparator. Note also that convolution here operates over *time*, not over space as in [3], [5] and [15]: this is explicitly not a spatio-temporal model.

2.5 Evaluation methodology and the case for strong baselines

The metric choice rests on Hyndman and Koehler [16], who show that many widely used accuracy measures "are degenerate in commonly occurring situations", specifically when actual values approach zero — which is exactly the situation in a 10-minute cellular series at night, and independently the reason [3] aggregated to hourly. Their recommendation, adopted here, is the mean absolute scaled error, whose denominator is the in-sample one-step naive error, making $MASE < 1$ directly interpretable [16], [11].

The decision to treat naive baselines as serious competitors is evidence-based rather than rhetorical. Makridakis *et al.* [21] found that on 1,045 monthly M3 series, machine-learning methods were "dominated across both accuracy measures used and for all forecasting horizons examined" by standard statistical methods, at considerably greater computational cost. The M4 Competition [22] gives the more nuanced successor result, in which combinations of statistical and machine-learning methods led rather than either family alone. Within this domain, Wang *et al.* [6] report their graph model achieving "62.2, 19.7, 16.3, 13.2 [...] percent smaller MAE than NAIVE, ARIMA, LSTM, HW" — from which the ordering is the informative part: their Holt-Winters implementation outperformed their own LSTM, and ARIMA trailed it by only a few points.

Fairness requires stating the counter-argument. The M3 and M4 series are short and low-frequency, with on the order of a hundred observations each; the Milan cells provide thousands of high-frequency observations per series, conditions far more favourable to learned models. That is the dimension along which this study tests whether the [21] prior survives.

Finally, two methodological tools and one limitation. STL [17] is used for decomposition because it allows the seasonal component to evolve, which matters across a window containing Christmas; component strengths follow the measures originating with Wang, Smith and Hyndman [18] in the form given by [11]. Training uses the Huber loss [26] for robustness to the traffic spikes documented in Section 4.4 and the Adam optimiser [27], the same optimiser used by [3] on this dataset. The limitation is raised by DeepCog [8]: symmetric error metrics do not correspond to operational value, since over- and under-provisioning carry asymmetric costs. Bega *et al.* train against an operator-defined asymmetric cost instead and report "substantial (50% or above) reduction of operating expenses" relative to allocation driven by conventional predictors [8] — a reduction in operating expense under their cost model, not in forecast error. This report uses symmetric metrics as the assignment requires, and Section 7 notes the gap.

2.6 Operational motivation

The energy argument in Section 1 rests on two independent accountings of where mobile-operator energy is consumed. A peer-reviewed survey by Larsen *et al.* [24] attributes 73% of network energy to the radio access network, while GSMA Intelligence's operator benchmark [23] — industry grey literature rather than peer-reviewed work — reports 87% for participating operators. The range 73–87% is quoted rather than a single figure. Larsen *et al.* further estimate that available technologies could cut overall mobile network energy consumption by roughly 30% [24], which bounds the prize. The mechanism connecting that prize to forecasting is base-station sleep-mode scheduling, surveyed by Wu *et al.* [25], which "takes advantage of changing traffic patterns" to idle lightly loaded stations — a decision taken on *anticipated* rather than observed load. That survey is also candid that simplifying assumptions in the literature "may lead to noticeably lower benefit" than models ignoring load-dependent power effects suggest [25], so the claim made here is the modest one: accurate short-horizon forecasts are a necessary input to such control, not a guarantee of a particular saving.

3. Dataset and Data Preparation

3.1 The data

The dataset is derived from Call Detail Records collected by Telecom Italia and released for the Big Data Challenge 2014, documented by Barlacchi *et al.* Milan is partitioned into a regular 100×100 grid; for each area, each 10-minute interval and each counterpart country code, the data records SMS activity (in/out), call activity (in/out), and Internet activity. The observation window runs from 1 November 2013 to 1 January 2014 inclusive: 62 days and therefore $62 \times 144 = 8,928$ intervals.

Activity values are a normalised proxy for volume rather than a physical unit; the publishers scaled them to protect commercial confidentiality. This has one methodological consequence worth stating early: absolute error magnitudes are not interpretable in bytes, so all comparisons in this report are made *between models on the same area*, and scale-free metrics are reported alongside absolute ones.

This study forecasts **Internet activity**, the target specified by the assignment and the channel most relevant to capacity planning.

3.2 The computational constraint

All work was carried out on an Intel Core i5-6300U (2 physical cores, 4 threads, 2.4 GHz) with 7.9 GB of RAM running Windows 10, with no GPU. The raw dataset is 19.38 GB across 62 tab-separated files and contains 319,896,289 rows. The dataset is therefore roughly 2.5 times the machine's total memory and, more importantly, more than an order of magnitude larger than the memory that was actually free.

This rules out the obvious approach outright. It is not a matter of efficiency: `pd.read_csv` on the whole dataset does not run slowly, it fails. The pipeline had to be designed so that peak memory is a function of a chosen chunk size rather than of the dataset size.

3.3 Memory management strategy

Five decisions do the work. They are listed in the order they take effect, with what each one buys.

1. **Aggregate away the country-code dimension during parsing.** Each raw row is a `(square, interval, country_code)` triple. The forecasting task is defined per geographical area, so the country dimension is summed out as each chunk is read rather than after loading. This reduces 319,896,289 rows to $62 \times 144 \times 10,000 = 89,280,000$ cells, a factor of 3.6, and it is the single largest reduction available because it removes a dimension rather than compressing one.

2. **Project to the three needed columns.** `usecols=[0, 1, 7]` parses `square_id`, `time_interval` and `internet` only. The four SMS and call columns are never converted from text or allocated. The country-code column is also skipped: because the aggregation sums over it, its values are not needed at all.

3. **Stream each day in fixed-size chunks.** Each file is read in 1,000,000-row chunks and folded into a pre-allocated accumulator. This is the decision that makes peak memory independent of file size.

4. **Store the result densely as float32 and read it back memory-mapped.** A dense `(8928, 10000)` matrix is the right structure here, and this was checked rather than assumed: between 1,432,827 and 1,439,946 of the 1,440,000 possible cells are non-zero on every single day, so the grid is essentially complete. A sparse format would store indices alongside values and end up *larger*. At `float32` the store is **341 MB**, and because it is accessed through `numpy.memmap`, extracting one area's complete 62-day series touches only 35 KB of resident memory.

5. **Download, process, discard.** Files are fetched one at a time and deleted immediately after aggregation, so peak disk use is `store + one raw day = 721 MB` instead of 19.38 GB. Files that were already present locally are never deleted.

One further detail matters for correctness rather than size. Accumulation is performed in `float64` and cast to `float32` only once a day is complete. Summing thousands of small activity values directly into a `float32` accumulator would lose precision progressively, since each addition rounds. This way the stored value is *accurate to float32* rather than being the *result of float32* arithmetic — a distinction that costs nothing because the accumulator is transient.

Measured effect

Four loading strategies were benchmarked on the same day file (308 MB, 4,842,625 rows), each in a **separate subprocess**. Running them in one process would understate the differences, because CPython does not generally return freed heap pages to the operating system, so a peak measured after an earlier variant had already grown the heap would be misleading.

variant	rows	seconds	baseline_rss_mb	peak_rss_mb	peak_above_baseline_mb	in_memory_payload_mb	raw_file_mb	reduction_vs_naive_%
naive	4842625	9.79	75.3	408	332.7	295.6	307.9	0
typed	4842625	14.8	75.5	273.9	198.4	157	307.9	32.9
projected	4842625	7.15	75.3	165.2	89.9	73.9	307.9	59.5
streamed	4842625	7.31	75.6	148.9	73.2	11	307.9	63.5

Peak resident memory falls from 408 MB to 149 MB, a factor of 2.7. The more telling column is `in_memory_payload_mb`: the data structure itself shrinks from 295.6 MB to 11.0 MB, a factor of 27. The two numbers differ because peak RSS includes a roughly 75 MB baseline for the interpreter and imported libraries, which no amount of optimisation removes.

The decomposition shows where the saving comes from. Declaring narrow dtypes alone removes 33%; adding column projection takes it to 60%; chunked streaming adds the last 4 percentage points of RSS but is responsible for the payload dropping to 11 MB. That last distinction is the important one, because it is the only change whose benefit *scales*: the payload of the first three variants grows with the file, while the streamed payload is fixed by the chunk size. Processing the full dataset the naive way would require on the order of $62 \times 296 \text{ MB} \approx 18 \text{ GB}$ of resident memory; the streaming path needs 11 MB regardless of how many days are processed.

Across all 62 days, measured peak RSS averaged 166 MB and never exceeded **241 MB**, against a nominal budget of 1 GB. Whole-dataset arithmetic:

quantity	value
Raw dataset on disk (62 tab-separated day files)	19.38 GB
Raw rows parsed (square x interval x country code)	319,896,289
Rows after summing out country code	89,280,000
Row-count reduction factor	3.6x
Compact store, float32 dense matrix	341 MB
Same store as float64	681 MB
Same store retaining all 5 activity types (float32)	1703 MB
Disk footprint reduction (raw -> store)	58x
Peak disk needed by the pipeline (store + 1 raw day)	721 MB
Resident memory to read one area's full series	35 KB

Trade-offs and limitations

The optimisation is deliberately one-directional, and it is worth being explicit about what was given up.

- **Only Internet activity is retained.** Keeping all five activity channels would make the store 1.70 GB. The cost is that a multivariate model using voice or SMS traffic as exogenous inputs would require re-downloading and re-parsing 19.38 GB. This was judged acceptable because the task specifies Internet traffic, but it is a real restriction on follow-up work.
- **The country-code breakdown is irrecoverable.** Roaming or international-traffic analyses are foreclosed by the same argument.
- **float32 is not lossless.** About seven significant decimal digits survive. This is far below the noise floor of an activity index that has itself been rescaled by the publisher, but the store is not a bit-exact copy of the source.
- **The dense layout assumes a complete grid.** This was verified for Milan; for a ragged or sparsely instrumented grid the same choice would waste space.
- **Download throughput needed engineering, not just patience.** Harvard Dataverse throttles an individual connection to roughly 0.17 MB/s for this dataset, while the host link sustained 5.3 MB/s against a reference server. Sequential download would have taken about 34 hours. Issuing eight concurrent HTTP range requests

per file achieved 4.0–5.0 MB/s, a 23-fold improvement, completing the transfer in about two hours. The worker count is configurable and deliberately modest, since the archive is a shared public resource.

3.4 Access and reproducibility note

The dataset is public, but Harvard Dataverse attaches a download guestbook to it. Since Dataverse 6.10 the plain `GET /api/access/datafile/{id}` endpoint returns HTTP 400 unless a guestbook response accompanies the request, *even for an authenticated user*. The pipeline uses the documented alternative: it POSTs a guestbook response to the same endpoint with `?signed=true`, receives a short-lived signed URL, and downloads through that. Because the signed URL expires after about a minute, the downloader re-issues it on demand rather than caching it for the duration of a transfer. This is recorded here because it is the single most likely obstacle to reproducing the work.

3.5 Data quality verification

Four checks were run on the assembled store before any modelling, on the principle that a silent indexing error would produce plausible and entirely wrong results.

1. **Completeness.** All 62 expected days are present in the manifest; no interval is missing; there are no NaNs; 0.019% of cells are exactly zero; no area has zero total traffic; and **zero rows** fell outside the expected time or square ranges across all 319,896,289 parsed rows.
2. **Timezone alignment.** The city-wide mean profile reaches its minimum at 05:00 and its maximum at 13:00 local time, and mean weekend activity is 0.83 of the weekday level. Both are what human activity should produce. Had the assumed CET offset been wrong, the profile would have been visibly shifted; this check is the reason the offset is stated as verified rather than assumed.
3. **Grid orientation.** Cell centroids from the official `milano-grid.geojson` correlate with `(id - 1) // 100` in latitude at $r = 1.000$ and with `(id - 1) % 100` in longitude at $r = 1.000$. The row index therefore runs south-to-north and the column index west-to-east, so the maps in Section 4 are north-up. A silent vertical flip here would have inverted every geographic interpretation.
4. **Calendar plausibility.** 1 November 2013 — All Saints' Day, a public holiday in Italy — shows a city total of 82.5 M against roughly 110 M on ordinary weekdays. The data reproduces a known calendar effect that was not used in its construction.

4. Exploratory Analysis

The purpose of this section is not to characterise the data for its own sake but to derive the modelling decisions of Section 5 from evidence. Each subsection therefore ends with the consequence drawn from it.

4.1 Distribution of traffic across geographical areas

Distribution of total Internet traffic across 10,000 Milan areas (1 Nov 2013 - 1 Jan 2014)

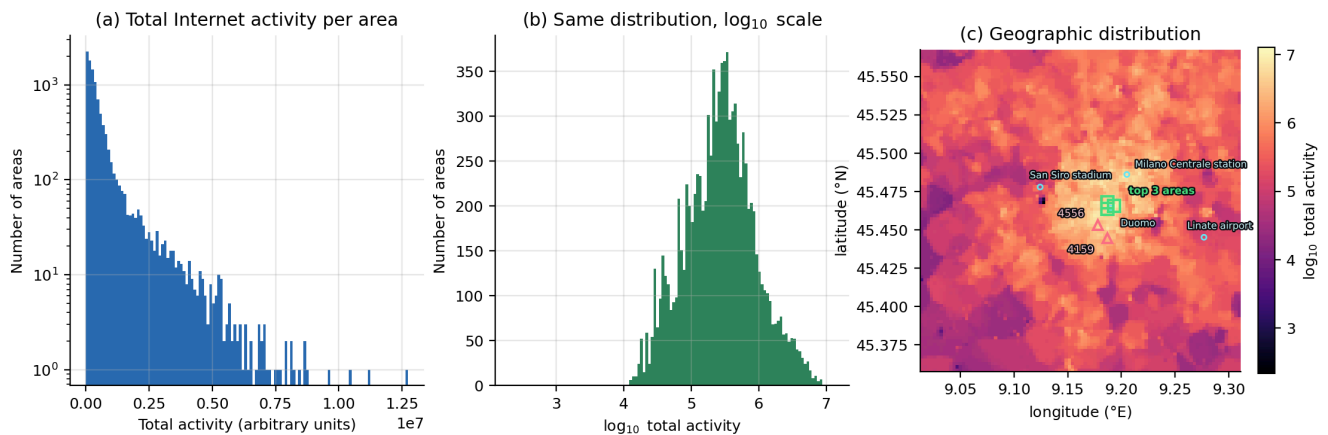


Figure 1. Total Internet activity per area over the full two months. (a) The raw distribution is extremely right-skewed. (b) The same distribution on a log scale is approximately unimodal, indicating a heavy-tailed rather than multi-modal population. (c) The geographically correct map, with the three highest-traffic areas (green squares) and the two reference areas 4159 and 4556 (pink triangles).

Traffic is distributed across the 10,000 areas with extreme inequality:

Statistic	Value
Mean / median total activity	555,289 / 277,871
Maximum + median	45.8
Skewness / excess kurtosis	4.26 / 25.50
Gini coefficient	0.608
Share of city traffic in the top 1% of areas	11.05%
Share in the top 10% of areas	48.39%
Share in the bottom 50% of areas	11.62%

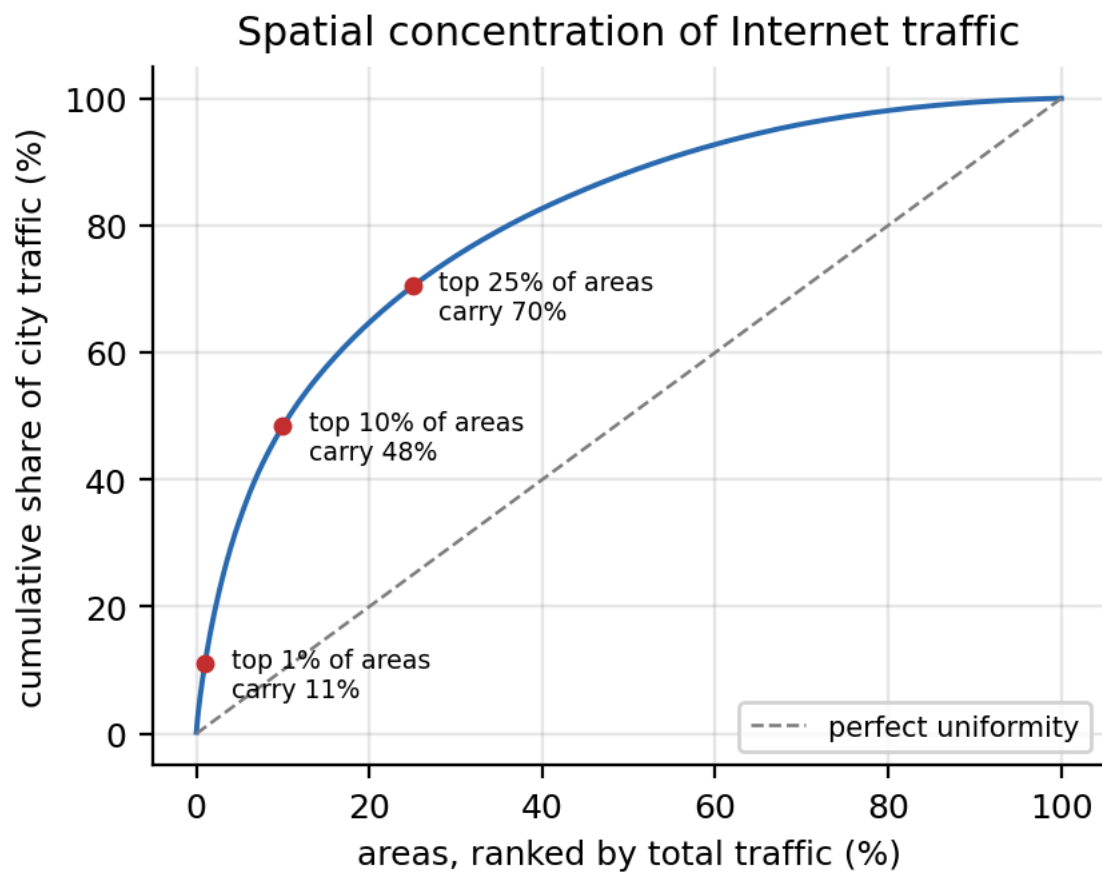


Figure 2. Cumulative share of city traffic against areas ranked by volume. The busiest 10% of areas carry as much traffic as the remaining 90% combined.

Panel (c) of Figure 1 explains the inequality. Traffic is spatially organised, not randomly scattered: it decays smoothly outwards from a single bright core, with secondary concentrations at recognisable infrastructure. The three highest-traffic areas are contiguous cells in the historic centre, all within 0.5 km of the Duomo.

Consequence. A conclusion drawn from one area cannot be assumed to hold city-wide. This justifies evaluating on three areas, and it is why the tuned configuration is chosen on one area and then applied *unchanged* to the others — so that cross-area differences measure generalisation rather than per-area fitting.

4.2 Traffic time series of specific geographical areas

Internet traffic in five Milan areas, first two weeks

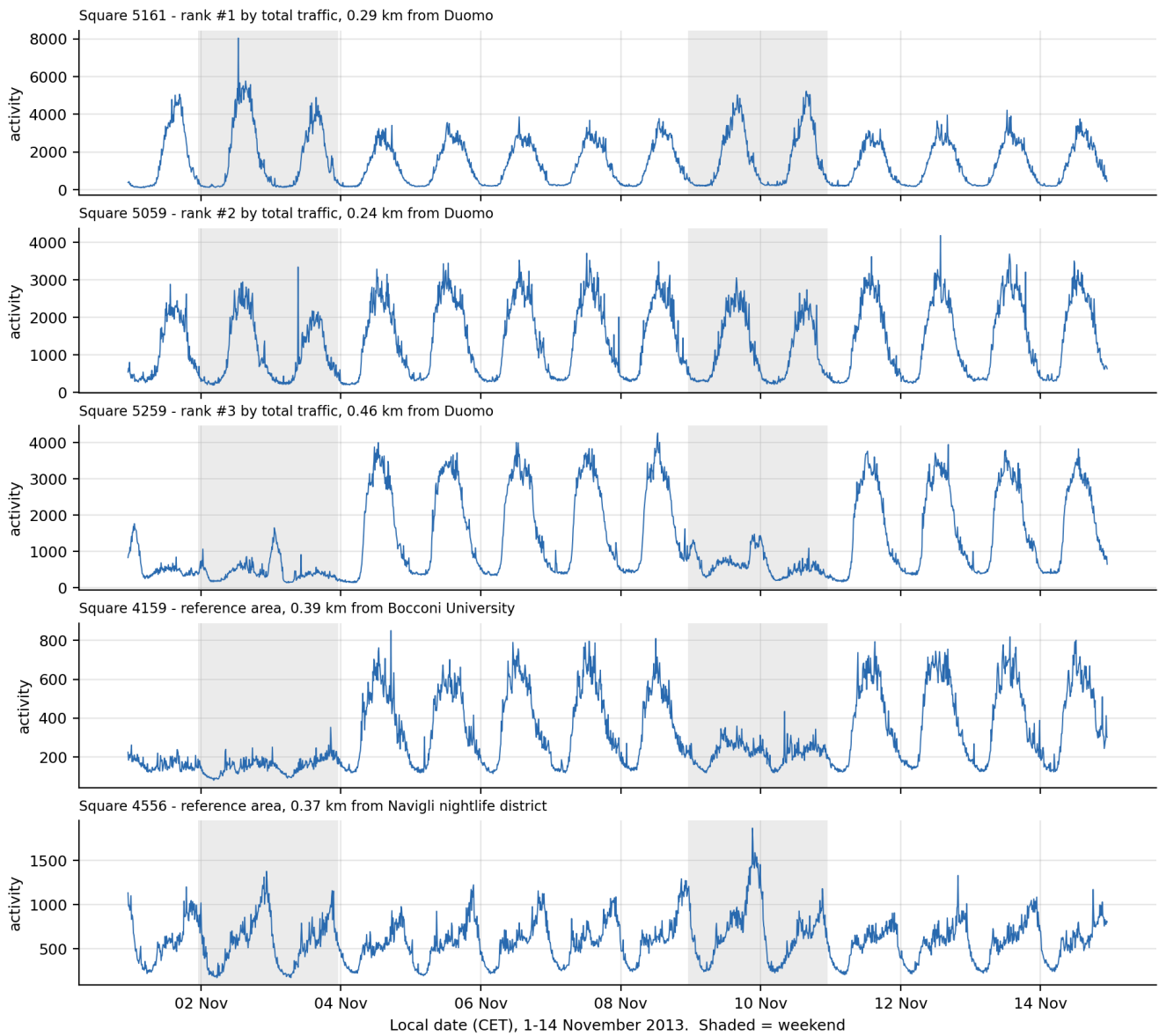


Figure 3. Internet traffic in the three highest-traffic areas and the two reference areas, 1–14 November 2013. Weekends shaded. Note that 1 November is All Saints' Day, a public holiday.

square	role	total	mean	std	cv	min	max	peak_over_trough	weekday_mean	weekend_mean	weekend_ratio	peak_hour
5161	Square 5161 - rank #1 by total traffic, 0.29 km from Duomo	1.274e+07	1427	1382	0.968	55.795	8044	45.641	1283	1777	1.385	16
5059	Square 5059 - rank #2 by total traffic, 0.24 km from Duomo	1.117e+07	1251	961.472	0.768	108.987	4666	16.486	1304	1123	0.861	14
5259	Square 5259 - rank #3 by total traffic, 0.46 km from Duomo	1.049e+07	1174	1103	0.939	72.542	4442	9.264	1412	599.634	0.425	13
4159	Square 4159 - reference area, 0.39 km from Bocconi University	2.454e+06	274.881	181.329	0.66	47.264	940.891	7.425	312.877	183.345	0.586	12
4556	Square 4556 - reference area, 0.37 km from Navigli nightlife district	4.575e+06	512.396	248.267	0.485	80.247	1869	7.28	492.786	561.34	1.139	22

The five areas differ by a factor of five in mean level, but the more useful observation is that they differ in *shape*. Normalising each series by its own mean makes this explicit:

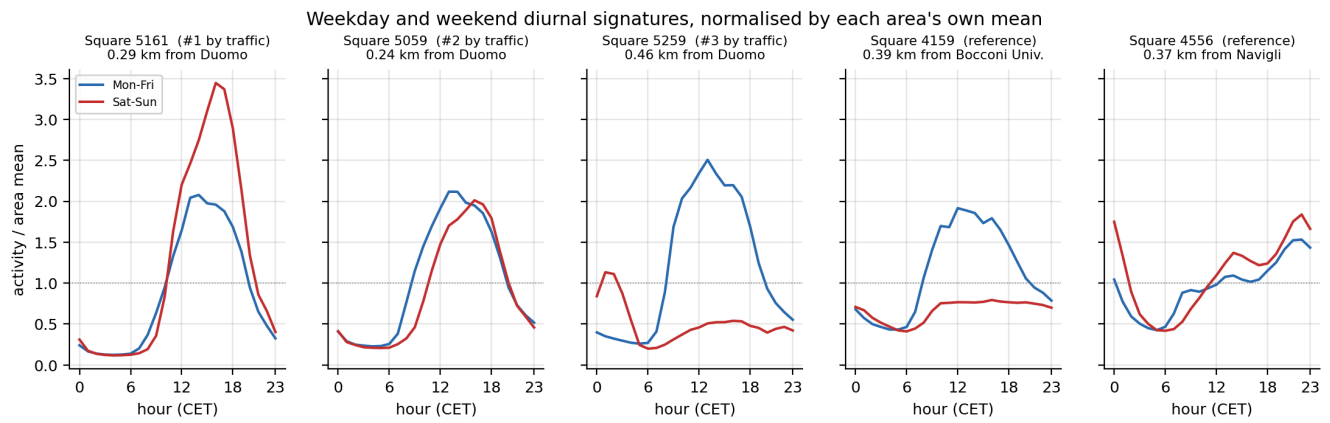


Figure 4. Weekday and weekend diurnal signatures, each normalised by that area's own mean. The shapes are qualitatively different, not merely rescaled.

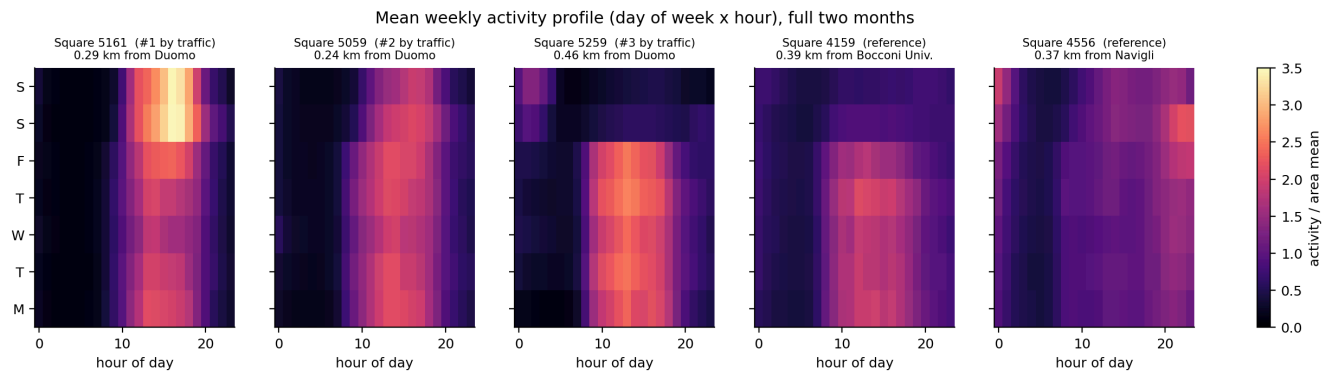


Figure 5. Mean day-of-week $\times$ hour profile per area, normalised by each area's mean, on a shared colour scale.

square	nearest_landmark	distance_km	mean	late_night_ratio	office_hours_ratio	night_over_office	weekend_over_weekday	weekday_peak_hour	weekend_peak_hour
5161	Duomo (historic centre)	0.29	1427	0.327	1.792	0.182	1.384	14	16
5059	Duomo (historic centre)	0.24	1251	0.45	1.706	0.264	0.861	13	16
5259	Duomo (historic centre)	0.46	1174	0.553	1.674	0.33	0.425	13	1
4159	Bocconi University	0.39	274.881	0.722	1.452	0.498	0.587	12	16
4556	Navigli nightlife district	0.37	512.396	1.326	1.032	1.285	1.14	22	22

The two reference areas specified by the assignment turn out to be well-chosen, and their behaviour is explicable from their location:

- **Square 4159**, 0.39 km from the **Bocconi University** campus, has an office-hours ratio of 1.45, a weekday peak at 12:00, and weekend activity only 0.59 of weekday activity. Figure 3 shows it flat and low through the 1–3 November holiday weekend, then switching to a strong daytime pattern the moment term activity resumes on Monday 4 November. This is a campus.
- **Square 4556**, 0.37 km from the **Navigli** nightlife district, is the inverse: late-night activity is 1.33 times its own daily mean, its night-to-office ratio is 1.29 — the only area above 1 — and it peaks at 22:00 on both weekdays and weekends, with weekend activity *above* weekday (ratio 1.14).
- **Square 5161** (Duomo, rank 1) peaks in the afternoon with weekend traffic 1.38 times weekday, consistent with retail and tourism, whereas **square 5259** just 0.2 km away is strongly weekday-dominated (weekend ratio 0.43), consistent with offices. Two adjacent cells in the same district behave oppositely.

Consequence. Because the *shape* differs between areas while the *scale* differs even more, scaling statistics are fitted per area rather than globally. And because the shapes differ, a single architecture transferring well across all three areas is a substantive result rather than a foregone conclusion.

4.3 Temporal dependence, periodicity and autocorrelation

Temporal dependence structure, square 5161

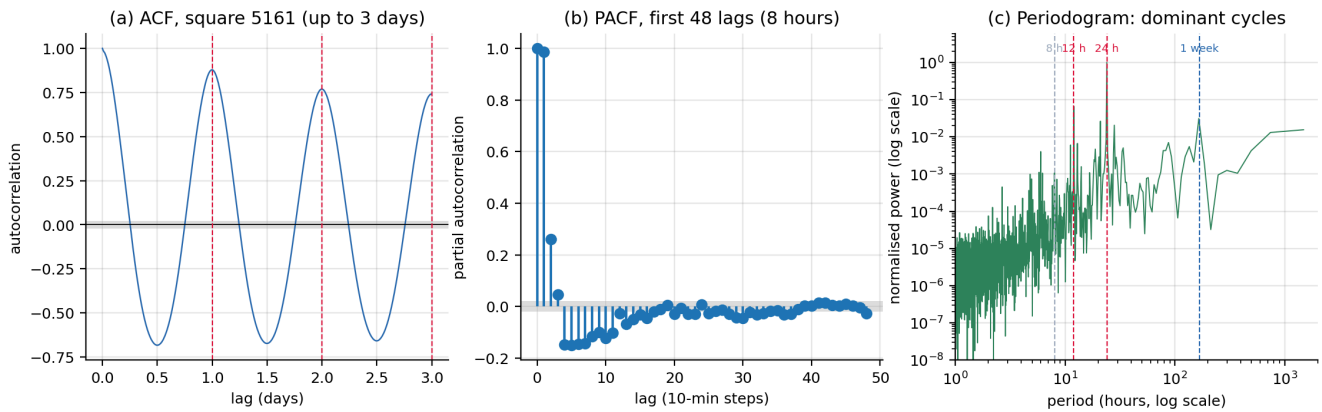


Figure 6. Square 5161. (a) ACF to three days: a clean sinusoid with maxima at exact one-day multiples and minima at half-day offsets. (b) PACF over the first 48 lags. (c) Periodogram on log-log axes; a linear power axis renders everything except the 24-hour peak invisible.

lag_label	lag_steps	acf
10 min	1	0.987
1 hour	6	0.939
6 hours	36	0.009891
12 hours	72	-0.683
1 day	144	0.878
2 days	288	0.77
3 days	432	0.741

Three quantitative facts follow, and each one is load-bearing.

The immediate past dominates. Autocorrelation at lag 1 (10 minutes) is 0.987 and at lag 6 (1 hour) is 0.939. Consecutive observations are nearly identical, which is why persistence must be treated as a serious baseline rather than a formality.

There is a strong, exactly 24-hour seasonality. Autocorrelation at lag 144 is 0.878, and at lag 72 (12 hours) it is **-0.683** — strongly negative, the day/night antiphase. The periodogram's dominant peak is at exactly 24.000 hours, with a harmonic at 12.000 hours at 6.6% of its power.

There is a weaker weekly component. The third-strongest interpretable spectral peak is at 165.3 hours — 6.9 days — at 3.1% of the daily peak's power. Autocorrelation remains high at two and three days (0.770, 0.741), decaying slowly.

The PACF is equally informative: it is 0.99 at lag 1, drops to 0.26 at lag 2 and 0.04 at lag 3, then oscillates within a narrow band. A short autoregressive order is therefore sufficient once seasonality has been removed.

Consequence. A candidate lookback of 144 steps is motivated by the lag-144 ACF peak: a model whose input window is shorter than one day *cannot* use "same time yesterday". For the TCN this is not a preference but a hard architectural constraint, since its receptive field is fixed by depth and kernel width — which is why receptive field is the first thing tuned in Section 5.4. The PACF bounds the SARIMA search to low orders.

4.4 Stationarity and decomposition

series	ADF_stat	ADF_p	ADF_verdict	KPSS_stat	KPSS_p	KPSS_verdict	std
raw	-19.032	0	stationary	0.237	0.1	stationary	1382
log1p	-22.743	0	stationary	0.571	0.026	non-stationary	1.203
first difference	-15.16	6.603e-28	stationary	0.003337	0.1	stationary	221.16
seasonal difference (lag 144)	-11.833	7.909e-22	stationary	0.069	0.1	stationary	639.05
log1p + seasonal difference	-9.956	2.444e-17	stationary	0.153	0.1	stationary	0.417

These results need careful reading, and the apparent contradiction in the first row is the interesting part. On the raw series both ADF and KPSS report stationarity. That is **not** evidence that the series is well-behaved: ADF tests for a unit root and KPSS for level stationarity, and *neither is sensitive to deterministic seasonality*. A series that cycles reliably between a 05:00 trough and a 14:00 peak has a strongly time-varying conditional mean while remaining mean-reverting around a stable level, which is exactly what these two tests are designed to accept. The seasonality is detected by the ACF and the periodogram in Section 4.3, not by these tests. Reporting the tests without this caveat would invite precisely the wrong conclusion.

The log1p row is the one that reveals structure: KPSS rejects level stationarity ($p = 0.026$) while ADF does not, the classic signature of a slow drift in level. After differencing at lag 144, both tests agree on stationarity with a much smaller KPSS statistic (0.069), and the same holds for the log1p + seasonal-difference combination (KPSS 0.153, ADF $p \approx 2 \times 10^{-17}$), whose standard deviation collapses to 0.417.

STL decomposition (daily period = 144 steps), square 5161

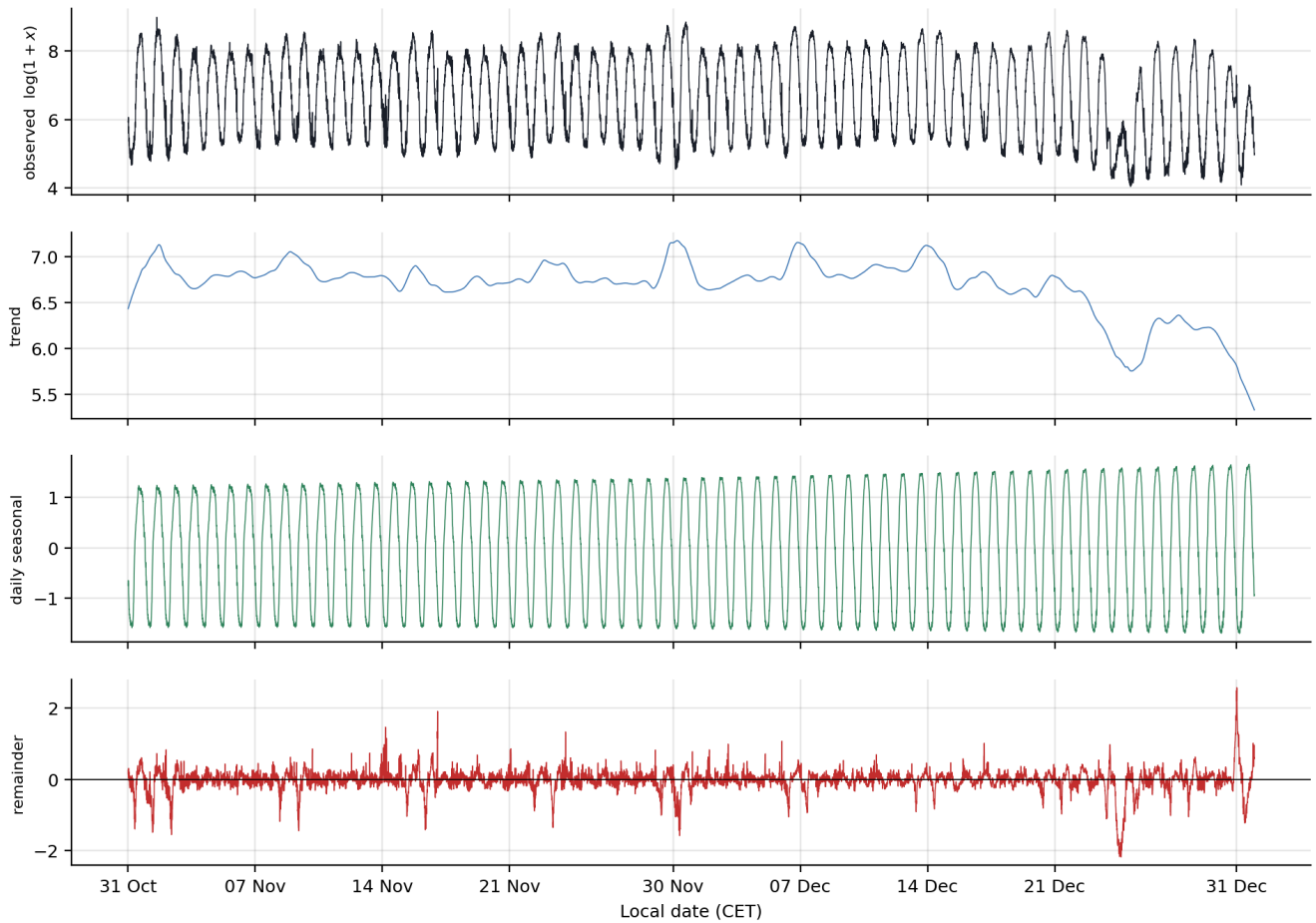


Figure 7. STL decomposition of $\log_{1p}$ activity for square 5161 with a 144-step daily period. The trend falls sharply after 21 December, the start of the Christmas holiday period.

quantity	value
Trend strength F_T (0-1)	0.519
Daily seasonal strength F_S (0-1)	0.927
$\text{Var}(\text{observed})$, log scale	1.447
$\text{Var}(\text{trend})$	0.092
$\text{Var}(\text{seasonal})$	1.289
$\text{Var}(\text{remainder})$	0.098
Remainder std as % of observed std	26.07

Component strengths follow Wang, Smith and Hyndman: each measures how much of the variation in "component + remainder" the component itself explains. Reporting $\text{Var}(\text{component}) / \text{Var}(\text{observed})$ would be misleading, because STL components are correlated and such ratios need not sum to 100%.

Daily seasonal strength is **0.927** — the daily cycle is the dominant structure — against a trend strength of 0.519. The remainder standard deviation is 26.1% of the observed standard deviation on the log scale. That figure is effectively an error floor: no model relying only on this area's own history can explain the irregular component, so it bounds what any of the models below can achieve.

Anomalies. 106 remainder observations exceed $|z| > 4$, and they are not noise — they are calendar effects:

Date	Anomalous slots	Sign	Interpretation
25 Dec 2013	50	negative (z to -6.85)	Christmas Day; retail district closed
1 Jan 2014	16	positive (z to +8.31)	New Year midnight celebration
31 Dec 2013	7	positive	New Year's Eve build-up
1–3 Nov 2013	14	negative	All Saints' Day holiday weekend

The largest single anomaly in the two months is 00:40 on 1 January, at $z = +8.31$. The strongest sustained departure is the whole Christmas Day afternoon.

Consequence. Seasonal differencing at lag 144 on the $\log_{1p}$ scale is the transform that makes this series stationary, which is a direct argument for $\text{SARIMA}(p,d,q)(0,1,0)$ [144] rather than an arbitrary order — the differencing is chosen by test, and the PACF then bounds p and q . Separately, the anomaly analysis identifies a genuine limitation of the experimental design: holiday behaviour is a large effect that no univariate model can anticipate, and the training period contains no comparable pre-Christmas period. The test week (16–22 December) precedes the worst of it, but the models are being asked to forecast a period whose calendar context is not represented in their training data.

5. Methodology

5.1 Forecasting task and evaluation protocol

The task is **one-step-ahead** forecasting: given observations up to interval t , predict activity at $t+1$, ten minutes later. This is evaluated over every one of the 1,008 intervals of the week **16–22 December 2013**, for each of the three highest-traffic areas (5161, 5059, 5259).

The data is split **chronologically and never shuffled**. Shuffling would place future observations in the training set, which invalidates any forecasting result.

Partition	Dates	Intervals	Role
Train	1 Nov – 8 Dec 2013	5,472	parameter estimation; scaler statistics
Validation	9 – 15 Dec 2013	1,008	hyperparameter selection; early stopping
Test	16 – 22 Dec 2013	1,008	reported results; read once

One point deserves care, because it is where forecasting evaluations most often leak. The validation and test predictions *do* condition on observations from immediately before them, including observations in an earlier partition. That is not leakage: at forecast time $t+1$, the value at t genuinely is available to an operator, and a model that refused to use it would be answering a different question. What never happens is the reverse — no target from validation or test influences a fitted parameter, a hyperparameter choice, or a scaler statistic. Concretely, the SARIMA coefficients are estimated on the training partition alone and then held fixed; the neural networks select weights by validation loss and never see the test week during training; and the scaler's mean and standard deviation are computed from training data only.

Tuning is performed on the **highest-traffic area only** (square 5161), and the selected configuration is then applied **unchanged** to all three areas. This costs some per-area accuracy but buys the ability to interpret cross-area differences as generalisation rather than as differences in tuning effort.

5.2 Input representation and preprocessing

Every model receives the same information, transformed the same way.

Transform. Raw activity x is mapped to $y = \log_{1p}(x) = \log(1 + x)$, then standardised to $z = (y - \mu) / \sigma$ with μ and σ estimated **on the training partition of that area only**. Predictions are inverted with $\hat{x} = \max(\expm1(\sigma \cdot \hat{z} + \mu), 0)$.

Three reasons for this choice, all traceable to Section 4:

- The distribution is heavy-tailed (skewness 4.26 across areas; isolated spikes within series). Under a squared-error objective on raw values, a handful of peak observations would dominate the gradient.
- The series is heteroskedastic — variability scales with the level — and $\log_{1p}$ stabilises it. This matters for SARIMA, which assumes constant error variance.
- $\log_{1p}$ rather than $\log$ because activity can be exactly zero (0.019% of cells), and $\log_{1p}$ maps $[0, \infty)$ onto $[0, \infty)$ without a special case.

The final clip at zero is not cosmetic. Because $\log_{1p}$ maps zero to zero, any scaled prediction below $-\mu/\sigma$ would otherwise invert to a *negative* traffic volume, which is physically impossible. This is enforced in the inverse transform and covered by a unit test.

Windowing. For the neural models, training examples are built by sliding a window of length L over the scaled series: input $(z_{t-L+1}, \dots, z_t)$, target z_{t+1} . Windows are produced as a strided view rather than a copy, so materialising 5,328 overlapping 144-step windows costs no additional memory until they are batched. L is a tuned hyperparameter; the ACF analysis motivates 144 (one day) as the principal candidate, with 36 (six hours) and 288 (two days) as the shorter and longer alternatives.

SARIMA does not use windows. It receives the seasonally differenced $\log_{1p}$ series directly, which is the representation its own structure requires.

Metrics. All errors are computed **in original activity units, after inverting the transform**, so no model gains an advantage from being evaluated on a scale where its errors look smaller.

Metric	Why it is reported
MAE	Required. Directly interpretable; robust to the spikes.
RMSE	Required. Penalises large misses, so it exposes peak-time failures.
MAPE	Required. Scale-free, therefore comparable across areas.
sMAPE	MAPE is unreliable here — night-time activity approaches zero, so a small absolute error becomes a huge percentage. sMAPE is at least bounded. Reported for completeness only; Hyndman and Koehler [16] argue against it, and it is not relied on for any conclusion.
MASE	A scaled error in the sense recommended by Hyndman and Koehler [16], and the measure this report leans on. Definition used here, which differs from theirs: the denominator is the seasonal-naïve MAE on the same evaluation partition , not the in-sample one-step naïve MAE of the original definition [16]. The variant is used because "same time yesterday" is the meaningful competitor for a strongly daily series, and because scaling by an error measured on the same week removes any level shift between partitions. The consequence is that $MASE < 1$ reads as "beat seasonal-naïve on this week" rather than "beat in-sample persistence"; values are therefore comparable across areas in this report but not directly comparable with MASE figures published elsewhere.
R ²	Familiar reference point.
RMSE + MAE	A diagnostic, not a score: a ratio well above 1 means the error is concentrated in a few large misses rather than spread evenly.

5.3 Model selection and justification

Three models were required to be "sufficiently different". They were chosen to differ in **model family** and in **how each aggregates history**, rather than being three variants of one idea.

Model	Family	History aggregation	Parameters grow with
SARIMA(p,d,q)(0,1,0)[144]	linear, statistical	explicit seasonal differencing + ARMA terms	$p + q$
LSTM	recurrent, neural	gated hidden state updated sequentially	hidden size ²
TCN	convolutional, neural	stacked dilated causal convolutions, parallel over time	channels ² × levels

SARIMA is included because the exploratory analysis argues *for* it rather than merely permitting it: seasonal differencing at lag 144 makes the series pass both stationarity tests, which is precisely the condition under which a linear ARMA model is appropriate. It is also the honest reference point for whether deep learning is needed at all.

LSTM is the architecture most frequently reported for cellular traffic prediction, and its gating mechanism is designed for exactly the situation here — a dependency at lag 144 that a plain RNN would struggle to propagate.

TCN was chosen as the structural opposite of the LSTM. Where the LSTM processes the window sequentially with an unbounded-in-principle state, the TCN convolves over all positions in parallel with a receptive field fixed by architecture. That contrast is what makes the comparison informative: if the two perform similarly, the recurrence is not doing anything special; if they differ, the reason should be traceable to receptive field or optimisation behaviour.

Baselines. Two are reported throughout, not as competitors but as calibration:

- **Persistence:** $\hat{x}(t+1) = x(t)$. Given the lag-1 autocorrelation of 0.987, this is expected to be strong, and any model that fails to beat it is not earning its complexity.
- **Seasonal naive:** $\hat{x}(t+1) = x(t+1-144)$, i.e. the same time yesterday. This is the MASE denominator.

5.4 Model specifications

Each specification below is self-contained: it states the structure, what the model receives, and how it is trained.

SARIMA

Structure. `SARIMA(p, d, q)(0, 1, 0)[144]` on `log1p` activity.

Implementation, and why it differs from the obvious one. Writing `SARIMAX(order=(p,d,q), seasonal_order=(P,D,Q,144))` in `statsmodels` with a non-zero `P` or `Q` produces a state-space model carrying on the order of 144 state variables. Every Kalman filter pass over ~8,000 observations then takes minutes, the optimiser needs many passes, and an order search needs many fits — which is not affordable on two CPU cores. The seasonal difference is therefore applied **explicitly**:

```
1. y_t = log1p(x_t)
2. z_t = y_t - y_{t-144}
3. fit ARIMA(p, d, q) to z
4. invert:  $\hat{x}_{t+1} = \text{expm1}(\hat{z}_{t+1} + y_{t+1-144})$ 
```

This is mathematically the same model as `SARIMA(p,d,q)(0,1,0)[144]`, but the state dimension now depends only on `max(p, q+1)`. It made the order search affordable and it makes the seasonal step visible in the code rather than hidden inside a state-space matrix. A unit test confirms the differencing and its inversion are exact to within floating-point tolerance.

Restriction, stated plainly. Setting `P = Q = 0` means no seasonal AR or MA terms. This is a real restriction on the model class, accepted for two reasons: the computational cost above, and the finding in Section 4.4 that seasonal differencing alone already brings both stationarity tests into agreement, leaving limited residual seasonal structure for such terms to capture. It is reported here as a limitation rather than presented as a full SARIMA.

Training. Coefficients are estimated by maximum likelihood on the training partition only, then held fixed. One-step-ahead predictions over validation and test are obtained by running the Kalman filter with those fixed coefficients and reading off `dynamic=False` predictions, so the forecast for `t+1` is conditioned on true observations up to `t` but no test data influences any parameter. This mirrors an operational deployment: refit rarely, observe continuously.

Deterministic term. `trend='n'` (no constant). This is worth stating because `statsmodels` treats `trend=None` as *its* default, which for `d = 0` silently includes a constant — an easy mistake to make and to then misreport. A constant on an already differenced series implies a deterministic linear trend in the original, which Section 4 does not support. The alternative was nonetheless tested explicitly in the order search.

LSTM

Structure. A single- or two-layer LSTM over the scaled window, followed by a linear head applied to the final hidden state, producing one scalar. Input is univariate: the window is shaped `(batch, L, 1)`.

Training. Adam; Huber loss; gradient-norm clipping at 1.0; `ReduceLROnPlateau` on validation MAE; early stopping on validation MAE with the best weights restored; a wall-clock budget per run (see Section 5.5).

Why Huber rather than MSE. The series contains isolated order-of-magnitude spikes (Section 4.4). Under MSE those few observations dominate the gradient and the model hedges by over-predicting ordinary traffic. Huber is quadratic near zero and linear in the tail, so the fit stays honest for the bulk of the data. The `RMSE ÷ MAE` column in the results is the diagnostic for how peak-dominated each model's error ends up being.

TCN

Structure. Stacked residual blocks, each containing two causal convolutions with dilation 2^i at level `i`, followed by a linear head reading the final time position. Causality is enforced by left-padding and then trimming the right-hand overhang, so no output can depend on a future input — a property pinned by a unit test.

Receptive field. With two convolutions per level, kernel `k` and `L` levels:

$$RF = 1 + 2(k - 1)(2^L - 1)$$

This is the key design quantity. With `k = 3`, four levels give `RF = 61` — under seven hours, which *cannot* reach the lag-144 daily dependency identified in Section 4.3 — while six levels give `RF = 253`, which covers it comfortably. Receptive field is therefore tuned before capacity, because it is a hard constraint rather than a soft preference.

Training. Identical to the LSTM: same loss, optimiser, scheduler, stopping rule and budget, so any difference in outcome is attributable to architecture rather than training procedure.

5.5 Experimental design for tuning

Hyperparameters were selected by a **documented sequential search**, not a grid. Each experiment changes one thing, and the reason for the next change is derived from the previous result. Every configuration tried, its validation scores, its cost, and the rationale for trying it, are appended to `results/tuning_<model>_sq5161.csv` as the search runs.

A grid search was not affordable and would not have been more informative. One LSTM epoch over the 5,328 training windows costs several seconds on this machine and one TCN epoch substantially more; a five-dimensional grid at 30 epochs per point would take days. More importantly, a grid would not have surfaced *why* a configuration works — whereas deliberately starting the TCN with an insufficient receptive field and watching it fail is direct evidence that the lag-144 dependency identified in the EDA is real and load-bearing.

Compute budget, and its consequence for interpretation. Tuning runs are capped at 30 epochs with patience 5 and a 420-second wall-clock budget per configuration; the finally selected configurations are retrained with 60 epochs, patience 8 and a 900-second budget. The cap has a methodological cost that must be acknowledged: a configuration is selected for being best *under this budget*, which is not the same as being best asymptotically. Where a run stopped because it hit the cap rather than because it converged, this is recorded in the experiment log and noted in the discussion.

Reproducibility. All random seeds are fixed (`RANDOM_SEED = 42`), PyTorch is pinned to two threads to match the two physical cores, and the same seed and thread count are used for every run so that timings are comparable.

6. Results and Discussion

6.1 Hyperparameter experiments

All tuning was performed on square 5161 and scored on the validation week (9–15 December). The complete logs, including the rationale recorded *before* each run, are in `results/tuning_*_sq5161.csv`. MASE is quoted alongside MAE because it makes the comparison against "same time yesterday" immediate.

SARIMA order search

experiment	rationale	model	square_id	partition	MAE	RMSE	MAPE_%	sMAPE_%	R2	RMSE/MAE	MASE	train_seconds	inference_seconds	inference_ms_f
1	Baseline: a single AR term on the seasonally differenced series.	SARIMA	5161	val	154.711	246.865	11.428	11.267	0.974	1.596	0.525	0.863	0.067	
2	A pure MA term at the same complexity, to see which the data prefers.	SARIMA	5161	val	222.226	378.251	14.606	14.286	0.938	1.702	0.754	0.662	0.101	
3	ARMA(1,1): the standard first upgrade when both ACF and PACF decay.	SARIMA	5161	val	136.333	210.59	10.19	10.066	0.981	1.545	0.462	0.938	0.096	
4	The PACF shows significant mass at lag 2; add a second AR term.	SARIMA	5161	val	137.011	211.411	10.247	10.123	0.981	1.543	0.465	1.765	0.12	
5	Probe whether AR order keeps helping or starts to overfit.	SARIMA	5161	val	137.023	211.242	10.254	10.129	0.981	1.542	0.465	3.91	0.24	
6	Trade an AR term for an extra MA term at equal parameter count.	SARIMA	5161	val	136.933	210.792	10.248	10.123	0.981	1.539	0.464	9.383	0.098	
7	Add a non-seasonal difference to test for residual drift.	SARIMA	5161	val	136.632	207.997	10.411	10.287	0.981	1.522	0.463	2.168	0.113	
8	Re-test the best order with a constant, to check whether the seasonally differenced series carries any residual mean offset. Worth testing explicitly because statsmodels includes a constant by default and it is easy to fit one unintentionally.	SARIMA	5161	val	137.045	211.415	10.251	10.123	0.981	1.543	0.465	4.906	0.464	

The search was bounded by the PACF of Section 4.3, which is 0.99 at lag 1, 0.26 at lag 2 and 0.04 at lag 3 — so only low orders were worth trying.

Experiment 2 is the informative failure. A pure MA(1) term scores 222.2 against AR(1)'s 154.7, and its AIC is worse by more than 1,600 units. This is the signature of a series whose dependence is autoregressive rather than shock-driven, which is what a PACF that cuts off sharply after lag 2 predicts. Adding one MA term *on top of* the AR term then produces the largest single improvement in the study (154.7 → 136.3), so both components matter, but in that order.

Beyond ARMA(1,1) the search flattens. Experiments 3 to 8 span validation MAE 136.3 to 137.0, a spread of 0.5%, while AIC keeps improving slightly and reaches its minimum at ARMA(2,2) (−1197.5 against −1161.7 for ARMA(1,1)). **The two criteria disagree, and the disagreement is the useful result:** the extra terms buy in-sample likelihood but nothing out of sample. AIC rewards fit to the training period; the validation week says the additional structure does not generalise. Selection therefore followed held-out MAE and parsimony, giving **ARMA(1,1) on the seasonally differenced log1p series — 3 parameters, validation MAE 136.3, MASE 0.462.**

Two secondary findings are worth recording because both were pre-registered questions rather than afterthoughts. Adding a non-seasonal difference (experiment 7) changed MAE by 0.3 while worsening AIC by 93, confirming there is no residual drift once the seasonal difference is applied. And adding a constant (experiment 8) changed nothing (137.0 versus 137.0), confirming the seasonally differenced series carries no mean offset — which is why `trend='n'` is correct and why the `statsmodels` default would have been a silent error rather than a harmless one.

LSTM sequential search

experiment	rationale	model	square_id	partition	MAE	RMSE	MAPE_%	sMAPE_%	R2	RMSE/MAE	MASE	train_seconds	inference_seconds	inference_ms_per
1	Start small: 6 hours of history, 32 units. Establishes whether short-range continuity alone already beats the baseline.	LSTM	5161	val	108.213	166.468	7.852	7.971	0.988	1.538	0.367	101.62	0.052	
2	Extend the window to a full day (144 steps). The ACF shows a strong peak at lag 144, so the model should be able to use yesterday's same-time value.	LSTM	5161	val	112.522	173.409	8.343	8.257	0.987	1.541	0.382	239.949	0.332	
3	Double the hidden width at the better lookback to test whether capacity, not context, is the binding constraint.	LSTM	5161	val	107.274	164.035	8.013	8.004	0.988	1.529	0.364	231.908	0.275	
4	Add depth with light dropout: a second layer can compose short-range and daily features hierarchically.	LSTM	5161	val	130.064	195.437	9.755	9.553	0.984	1.503	0.441	667.336	5.207	
5	Larger batches with a proportionally higher learning rate. Tests whether the earlier runs were optimisation-limited rather than capacity-limited, at roughly half the epoch cost.	LSTM	5161	val	113.964	171.545	9.248	8.992	0.987	1.505	0.387	455.831	1.961	
6	Two days of history, to check whether the weekly structure visible in the EDA is reachable through a longer window.	LSTM	5161	val	144.693	217.255	10.553	10.477	0.98	1.502	0.491	453.058	1.799	

The intended narrative was that extending the lookback from 36 steps (6 hours) to 144 steps (one day) would help, because the ACF has a strong peak at lag 144. Experiment 2 contradicted that: MAE went from 108.2 to **112.5**, worse.

That result is only explicable in combination with experiment 3. Doubling the hidden width at the same 144-step lookback recovered the loss and produced the best LSTM configuration of the study (**107.3, MASE 0.364**), while widening was what the earlier failure had made necessary. The reading is that **a longer window is not free**: 32 units spread over 144 timesteps had to encode four times as much history into the same state, and the daily-lag information it gained cost more in representational pressure than it returned. Context and capacity are not independent knobs here, and tuning them one at a time — the usual advice — initially pointed the wrong way. Only experiment 3 disambiguated them.

Experiments 4 to 6 must be reported with a caveat, because all three hit the 420-second wall-clock cap and are therefore **truncated, not converged**. The two-layer model reached only 6 epochs and scored 130.1; the two-day window reached 7 epochs and scored 144.7. These are not evidence that depth or long windows are intrinsically

bad — they are evidence that on this hardware they do not reach a usable state within an affordable budget. Under a fixed compute budget that distinction does not change the decision, and the budget is a real constraint rather than an artefact, but it does change what may be concluded, so the log records the truncation explicitly.

One further honest note: experiment 1 recorded its best epoch at 29 of 30, i.e. it was still improving when the epoch cap stopped it. Its 108.2 is therefore an upper bound on what the short-window model can do, and the margin over experiment 3 is narrower than the table suggests.

TCN: a search that its own budget invalidated

experiment	rationale	model	square_id	partition	MAE	RMSE	MAPE_%	sMAPE_%	R2	RMSE/MAE	MASE	train_seconds	inference_seconds	inference_ms_p
1	Receptive field 61 steps (~10 h): deliberately shorter than one day, to demonstrate that the daily lag is out of reach.	TCN	5161	val	103.172	158.36	7.701	7.681	0.989	1.535	0.35	408.731	0.393	
2	Two more dilation levels push the receptive field to 253 steps, which covers the lag-144 daily peak identified by the ACF.	TCN	5161	val	106.134	159.585	7.846	7.831	0.989	1.504	0.36	450.03	1.318	
3	With the receptive field sufficient, widen the channels to test whether representational capacity is now the limit.	TCN	5161	val	110.483	171.53	8.286	8.189	0.987	1.552	0.375	426.016	10.976	
4	Remove dropout: with only ~5k training windows the noise it injects may cost more than the regularisation it buys.	TCN	5161	val	106.098	163.7	7.604	7.758	0.988	1.543	0.36	445.883	2.568	
5	Receptive field 187 with one fewer dilation level, trading depth for kernel width. The deepest level dominates cost because it convolves the most heavily padded sequence, so this is markedly cheaper at a comparable receptive field.	TCN	5161	val	111.977	173.559	8.805	8.596	0.987	1.55	0.38	454.782	7.81	
6	Receptive field 509 over a two-day window, matching the longest-context LSTM configuration for a fair comparison.	TCN	5161	val	111.229	169.046	8.071	8.054	0.988	1.52	0.377	474.637	10.624	

Read naively, this table says the best TCN is the one with a **61-step receptive field** (103.2, the best validation MAE anywhere in the tuning phase) — the very configuration Section 4.3 predicted should be handicapped, because 61 steps cannot reach the lag-144 daily peak.

That conclusion does not survive inspection of the cost column. Experiment 1 is the *only* TCN run that stopped by early stopping; experiments 2 to 6 all hit the wall-clock cap, at 15, 5, 8, 9 and 5 epochs respectively. Per-epoch cost in a TCN grows sharply with depth, because the deepest dilated layer convolves the most heavily padded sequence, so the deeper models bought their extra receptive field by training for a third or a fifth as long. **The experiment confounded receptive field with training effort, and therefore could not answer the question it was designed to ask.**

This is the single most consequential methodological problem encountered in the project, and the remedy was to design a second experiment rather than to report the first one's ranking.

TCN controlled receptive-field study

The confound was removed by holding everything except depth fixed: identical channel width, identical batch size and learning rate, identical 144-step lookback, **an identical number of epochs for every configuration** (24), and early stopping disabled so that no run could terminate before any other. Depth then varies only the

receptive field: 61 steps (below the daily lag), 125 steps (just short of a full day), and 253 steps (covering the entire input window).

Two limits of this design should be stated before the numbers. First, each configuration was trained **once**, with the seed fixed at 42; the compute budget did not allow repeated runs, so run-to-run variance is unmeasured and only differences substantially larger than the spread already seen across configurations are interpreted below. What the fixed seed does establish is determinism: the four-level configuration reproduced its earlier result to the digit (103.17 in both studies), so the pipeline is reproducible even though seed sensitivity is unknown. Second, equalising *epochs* does not equalise *compute*: the deeper models still cost far more per epoch, and that cost is reported separately in Section 6.3 rather than being folded into the accuracy comparison.

experiment	rationale	model	square_id	partition	MAE	RMSE	MAPE_%	sMAPE_%	R2	RMSE/MAE	MASE	train_seconds	inference_seconds	inference_ms_per_s
1	Receptive field 61 steps (~10 h): cannot reach the lag-144 daily peak.	TCN	5161	val	103.172	158.36	7.701	7.681	0.989	1.535	0.35	1352	3.16	3.
2	Receptive field 125 steps (~21 h): just short of one full day.	TCN	5161	val	109.605	167.054	7.899	8.065	0.988	1.524	0.372	2110	0.972	0.1
3	Receptive field 253 steps (~42 h): comfortably covers the daily lag.	TCN	5161	val	104.546	159.839	7.723	7.747	0.989	1.529	0.354	987.646	1.062	1.1
4	Receptive field 509 steps (~85 h): covers two days, well beyond the window.	TCN	5161	val	102.473	154.714	7.694	7.647	0.99	1.51	0.347	1169	1.21	1.1

Validation MAE by receptive field: **61 → 103.17, 125 → 109.61, 253 → 104.55, 509 → 102.47.**

The expected result was a step improvement once the receptive field passed 144. That is not what happened, and the shape of the outcome is more informative than a confirmation would have been.

The ordering is non-monotonic. The 125-step model is worse than *both* of its neighbours, by 6.4 and 5.1 MAE units. Since depth is the only variable and each configuration trained for the same 24 epochs, nothing in the experimental design can explain a middle configuration being worst. The only available explanation is run-to-run variation, which therefore must be at least about 5 MAE units — and that figure is larger than the entire spread between the best and worst *monotonically ordered* configurations.

Against that yardstick, no receptive-field effect is detectable. The gap between the shortest field (103.17) and the longest (102.47) is 0.70 units, seven times smaller than the variation the experiment itself exposes. The conclusion is therefore a negative one, stated as such: **over the range 61 to 509 steps, receptive field has no measurable effect on one-step-ahead accuracy for this series.** A 61-step field that provably cannot see "same time yesterday" performs as well as one covering three and a half days.

This is not a contradiction of the EDA. It is a consequence of the *horizon*. The lag-144 ACF peak of 0.878 is real, but the lag-1 autocorrelation is 0.987, and at a one-step horizon the last few observations already carry almost all of the available signal. Yesterday's value adds little that this morning's values do not already imply. The daily cycle matters enormously for describing the series and for a linear model that must encode it explicitly — which is why seasonal differencing helps SARIMA — but a neural model conditioned on recent history does not need to reach back a full day to forecast ten minutes ahead. Section 6.4 returns to this, because it also explains why the models fail where they do.

Selection. Because the four configurations are statistically indistinguishable, the shallowest was selected: `levels = 4`, receptive field 61, 5,601 parameters, rather than the nominal best at `levels = 7` with 10,305 parameters. Choosing the lowest validation MAE here would have meant buying 84% more parameters and roughly double the per-epoch cost with a 0.7-unit difference as the justification. This is the same parsimony rule applied to the SARIMA order search, and it is implemented in `scripts/04_tune.py` rather than applied by hand, so the choice is reproducible and its threshold is derived from the data.

6.2 Final results on the test week

The tuned configurations — SARIMA(1,0,1)(0,1,0)[144]; LSTM with 64 hidden units over a 144-step window; TCN with 16 channels and 4 dilation levels — were applied **unchanged** to all three areas and evaluated on the 1,008 intervals of 16–22 December 2013.

Square 5161 (Duomo, highest total traffic):

model	MAE	MAPE_%	RMSE	sMAPE_%	MASE	R2	RMSE/MAE
Persistence	92.802	9.194	134.878	9.096	0.274	0.99	1.453
Seasonal naive	338.594	25.94	619.04	22.828	1	0.793	1.828
SARIMA	108.035	10.574	163.051	10.328	0.319	0.986	1.509
LSTM	93.118	8.364	147.011	8.175	0.275	0.988	1.579
TCN	78.492	7.496	119.578	7.367	0.232	0.992	1.523

Square 5059 (second highest):

model	MAE	MAPE_%	RMSE	sMAPE_%	MASE	R2	RMSE/MAE
Persistence	81.517	7.959	114.375	7.909	0.475	0.985	1.403
Seasonal naive	171.736	18.018	245.87	16.621	1	0.931	1.432
SARIMA	89.292	8.626	127.904	8.509	0.52	0.981	1.432
LSTM	67.231	6.451	97.879	6.42	0.392	0.989	1.456
TCN	68.867	6.874	99.478	6.73	0.401	0.989	1.444

Square 5259 (third highest, office-dominated):

model	MAE	MAPE_%	RMSE	sMAPE_%	MASE	R2	RMSE/MAE
Persistence	75.968	8.115	109.578	8.074	0.162	0.99	1.442
Seasonal naive	470.32	71.621	861.621	42.613	1	0.409	1.832
SARIMA	81.776	8.722	123.334	8.66	0.174	0.988	1.508
LSTM	67.321	7.214	96.814	7.151	0.143	0.992	1.438
TCN	64.745	7.112	93.3	6.934	0.138	0.993	1.441

The result that governs the interpretation

Seasonal naive is not merely beaten, it is **catastrophically** beaten: MAE 338.6, 171.7 and 470.3 against 92.8, 81.5 and 76.0 for persistence. On square 5259 its R² is 0.409 and its MAPE 71.6%. Every model therefore scores MASE far below 1, and reporting only MASE would make all five methods look like successes.

That is a trap, and avoiding it is the single most important interpretive step in this report. Because lag-1 autocorrelation is 0.987 while lag-144 is 0.878, **persistence — not seasonal naive — is the benchmark that actually binds**. Restating every result as a ratio to persistence changes the conclusions completely:

model	sq5059	sq5161	sq5259	mean	spread
Persistence	1	1	1	1	0
Seasonal naive	2.107	3.649	6.191	3.982	4.084
SARIMA	1.095	1.164	1.077	1.112	0.088
LSTM	0.825	1.003	0.886	0.905	0.179
TCN	0.845	0.846	0.852	0.848	0.0074

Three findings follow, and each is stronger than any single MAE number.

1. SARIMA never beats persistence. Ratios of 1.095, 1.164 and 1.076 — it is 7.6% to 16.4% *worse* on all three areas. This is not a tuning failure: the order search in Section 6.1 was bounded by the PACF, tested eight configurations, and flattened well before the boundary of the search space. It is a statement about the model class. A linear model with three parameters, given a seasonally differenced series, cannot match simply repeating the last observation, because the seasonal difference discards the very short-range information that dominates at a ten-minute horizon. Section 6.1's SARIMA study measured this indirectly already: differencing at lag 144 is what makes the series stationary and therefore linearly modellable, but stationarity is not the same as predictability.

2. The TCN's advantage is remarkably stable across areas. Ratios of 0.845, 0.846 and 0.852 — a 14.8% to 15.5% improvement on persistence, with a spread of **0.007** across three areas that differ by a factor of five in level, differ in weekend behaviour (5161 has weekend traffic 1.38× its weekday level, 5259 only 0.43×), and were characterised in Section 4.2 as having qualitatively different diurnal shapes. The configuration was tuned on 5161 alone and transferred unchanged. This is the most direct answer this study offers to the second half of its research question: for the TCN, the ranking and the magnitude of the gain both survive a change of location.

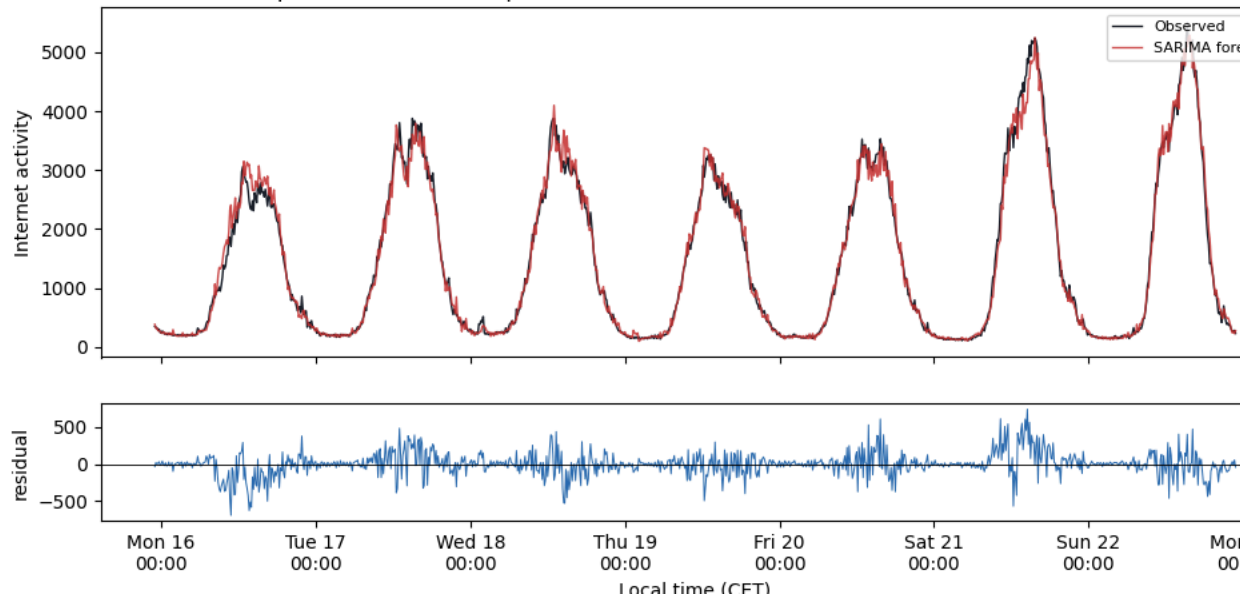
3. The LSTM's advantage does not transfer reliably. Ratios of 0.825, 1.003 and 0.886 — a spread of 0.179, twenty-five times the TCN's. It is the best model on square 5059 (MAE 67.2, better than the TCN's 68.9), and on square 5161 it merely ties persistence (93.1 against 92.8) while the TCN there achieves 78.5. Two models with nearly identical mean performance (0.905 against 0.848) therefore differ sharply in *reliability*, which a table of per-area MAEs alone would not reveal.

One honest caveat attaches to that comparison, and it happens to fall on the weakest LSTM result. The LSTM fit on square 5161 is the **only run in the final evaluation that hit its wall-clock budget**, stopping at epoch 22 of 40 with its best epoch at 18. Its best epoch matches the value found during tuning, so the fit had most likely converged, but this cannot be asserted: the one area where the LSTM fails to beat persistence is also the one area where it was not allowed to finish. The comparison on 5059 and 5259 is unaffected.

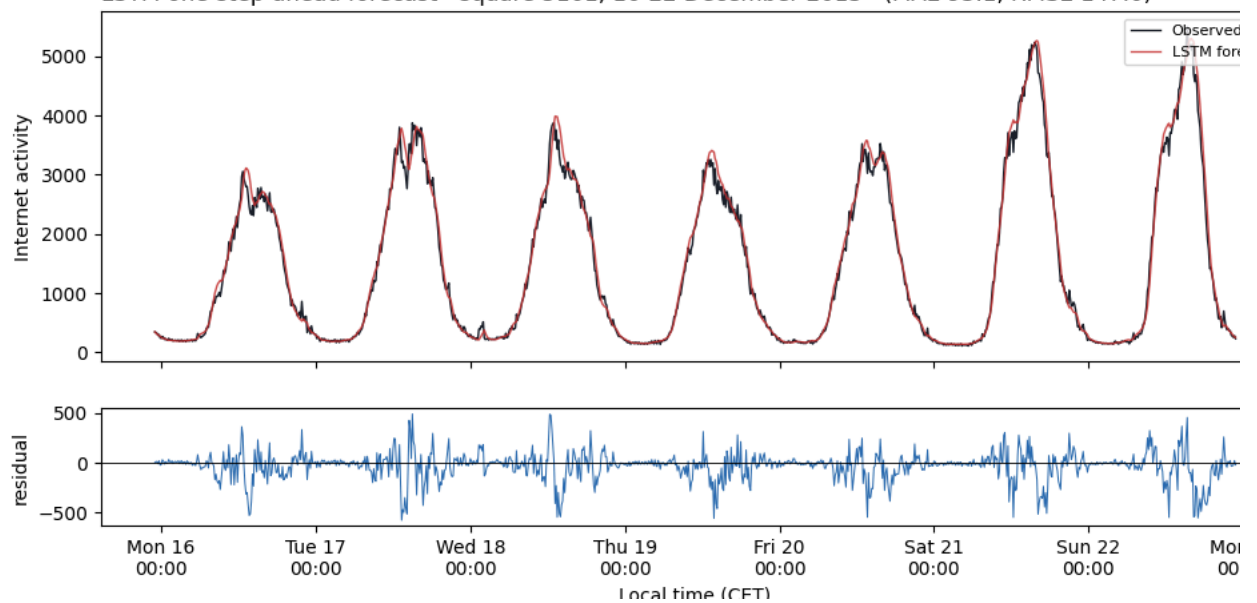
Forecast plots

The nine required plots are below, one per area and model, each with its residual series beneath. All share the same y-axis within an area. Times are local (CET).

SARIMA one-step-ahead forecast - square 5161, 16-22 December 2013 (MAE 108.0, RMSE 163.1)



LSTM one-step-ahead forecast - square 5161, 16-22 December 2013 (MAE 93.1, RMSE 147.0)



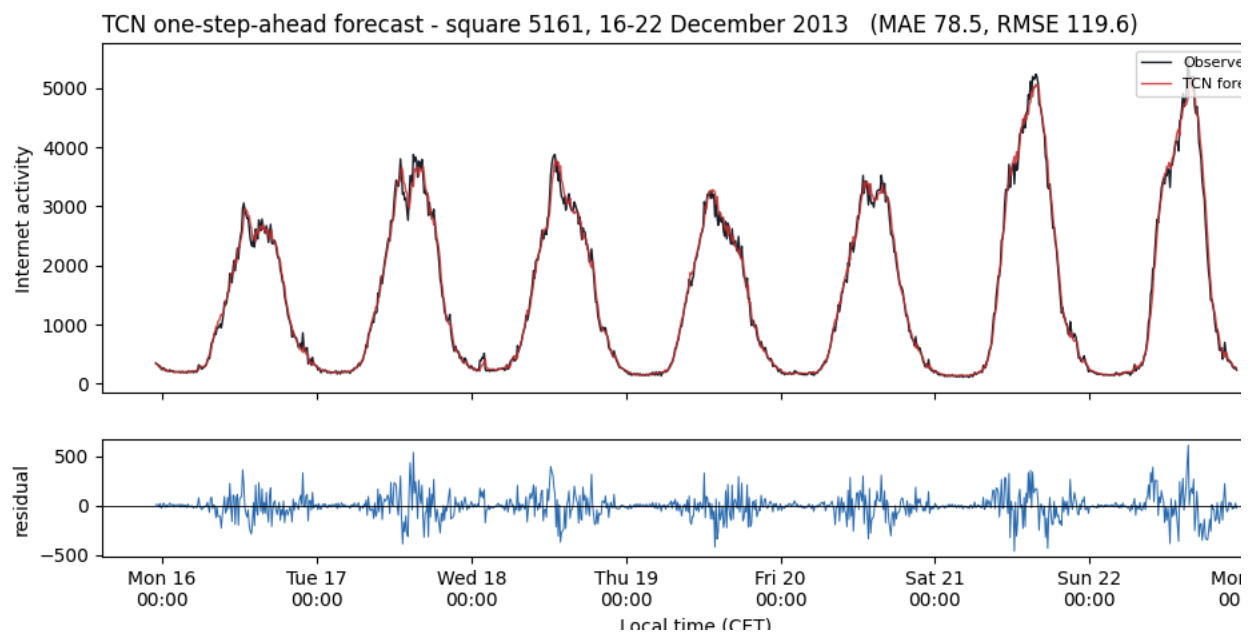
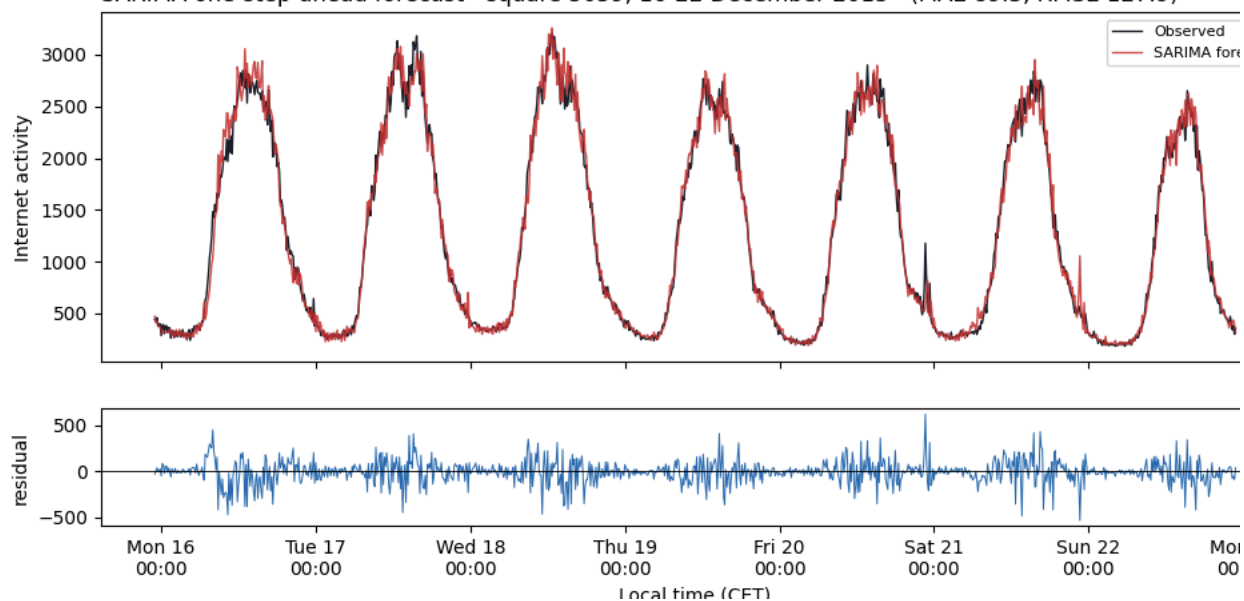
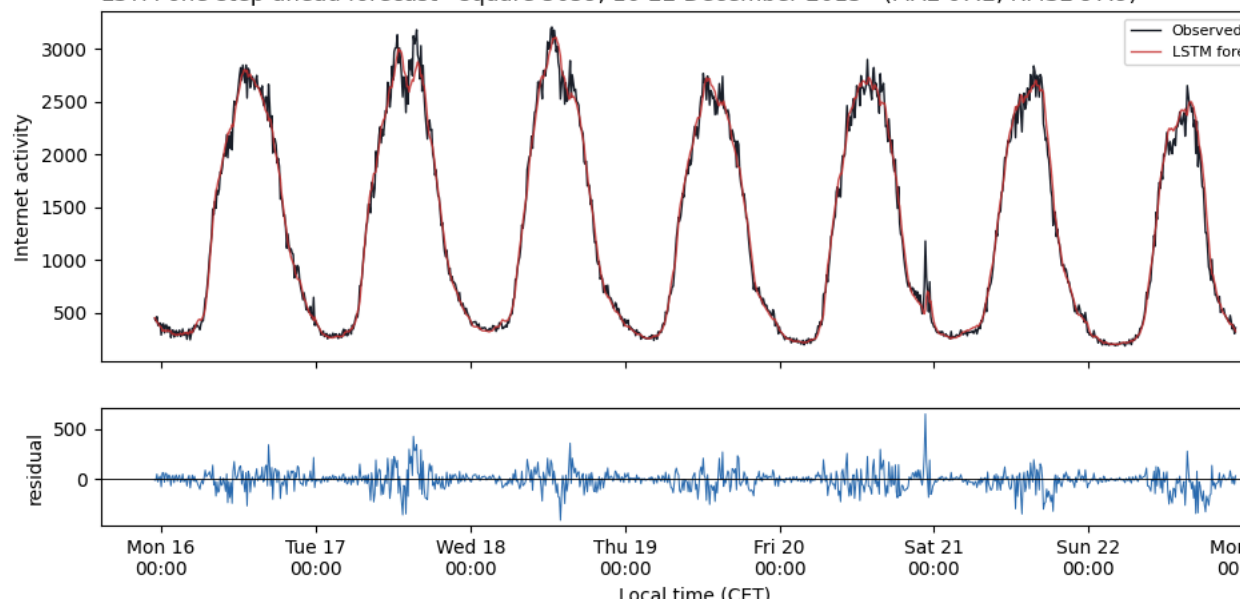


Figure 8. Square 5161, 16–22 December 2013: observed against one-step-ahead forecasts for SARIMA, LSTM and TCN, with residuals below each.

SARIMA one-step-ahead forecast - square 5059, 16-22 December 2013 (MAE 89.3, RMSE 127.9)



LSTM one-step-ahead forecast - square 5059, 16-22 December 2013 (MAE 67.2, RMSE 97.9)



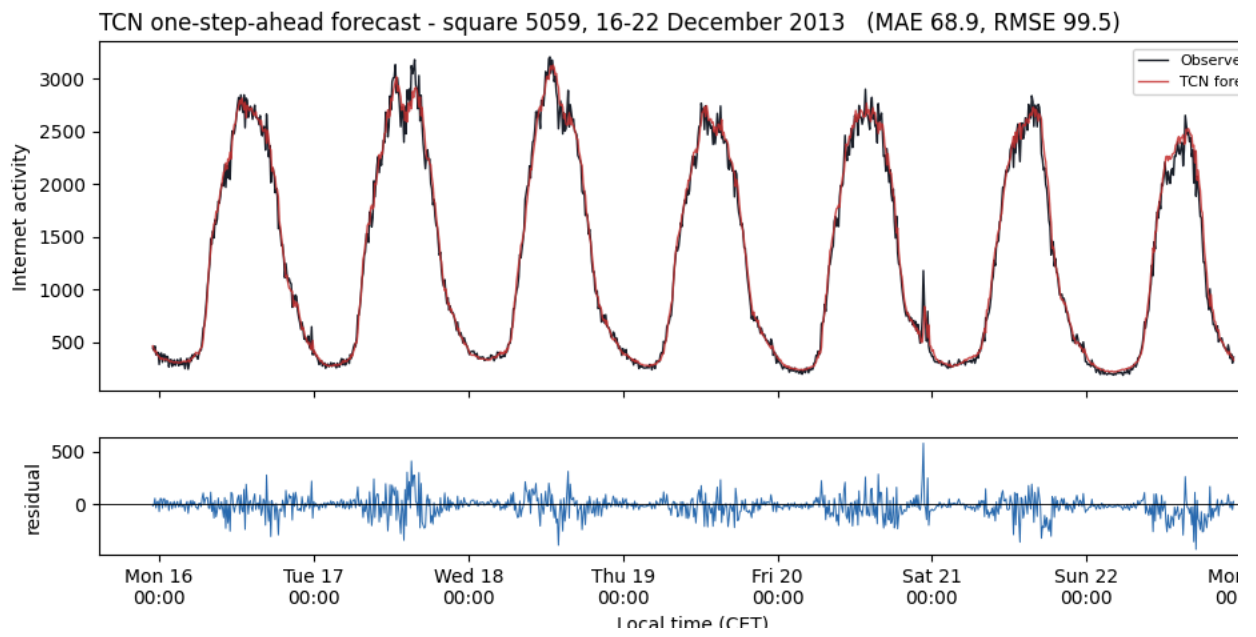
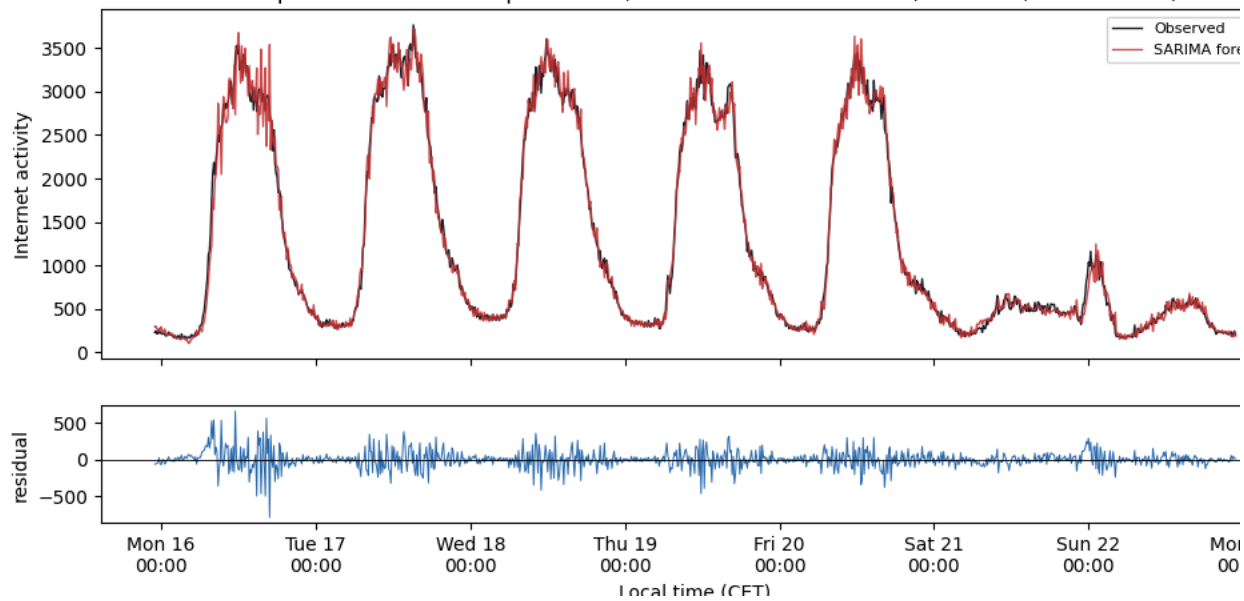
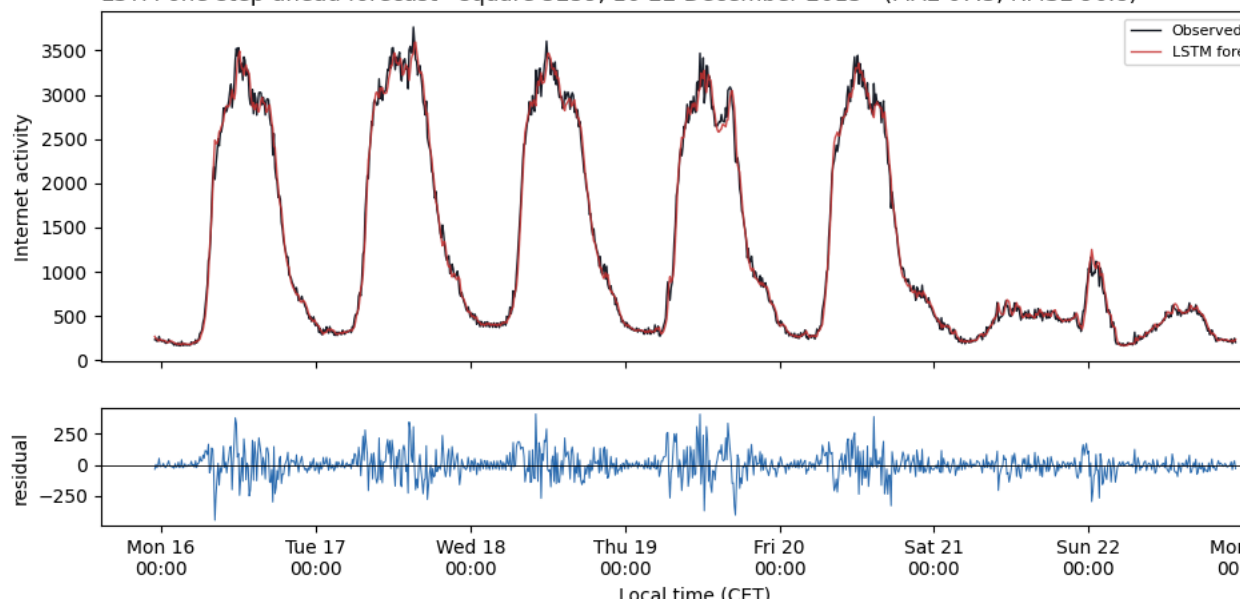


Figure 9. Square 5059, same week and models.

SARIMA one-step-ahead forecast - square 5259, 16-22 December 2013 (MAE 81.8, RMSE 123.3)



LSTM one-step-ahead forecast - square 5259, 16-22 December 2013 (MAE 67.3, RMSE 96.8)



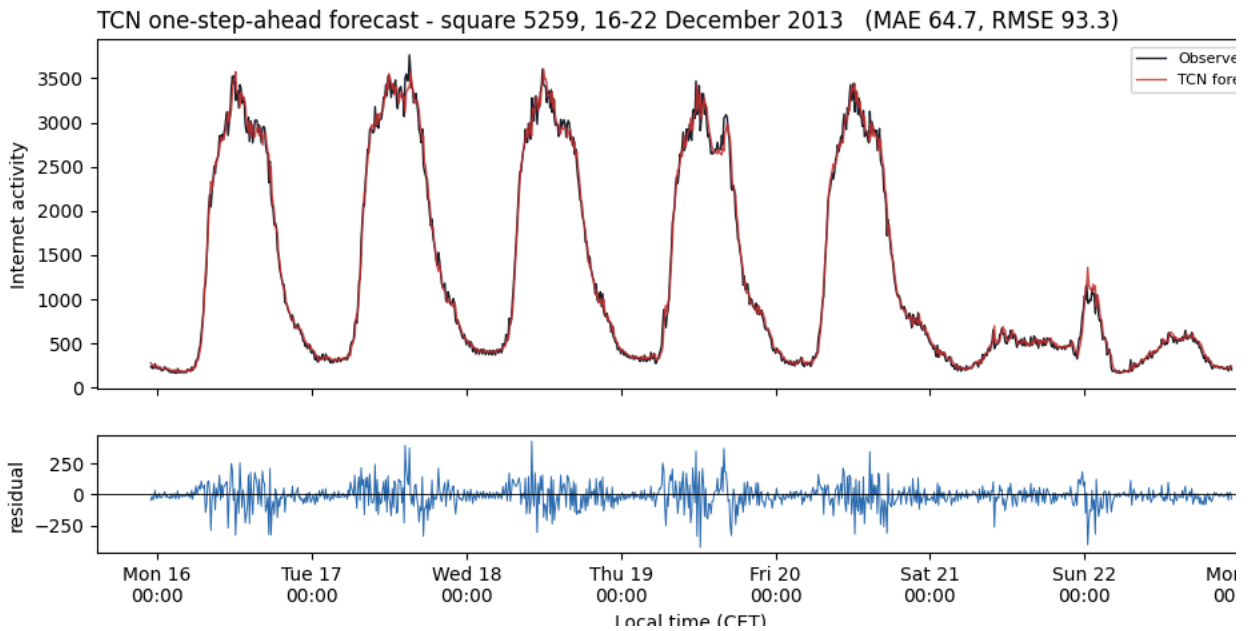


Figure 10. Square 5259, same week and models.

At this scale all three models track the diurnal cycle closely — every R^2 is between 0.981 and 0.993 — which is precisely why the plots must be read through the residual panels rather than the level series. The residuals are small and symmetric overnight and expand sharply around the daily peak, and they are visibly largest on Saturday 21 and Sunday 22 December, the pre-Christmas weekend, when square 5161 reaches roughly 5,000 activity units against a weekday peak near 3,500.

The $RMSE \div MAE$ column quantifies this: 1.44 to 1.58 for the three models, against 1.83 for seasonal naive on squares 5161 and 5259. A ratio meaningfully above 1 means error is concentrated in a few large misses rather than spread evenly, so peak periods dominate the aggregate. That is the failure mode examined in Section 6.4.

6.3 Computational cost

model	train_seconds_mean	train_seconds_min	train_seconds_max	inference_seconds_week	inference_ms_per_step	n_parameters	best_epoch
Persistence	0	0	0	0.065	0.065	0	-
Seasonal naive	0	0	0	0.066	0.065	0	-
SARIMA	1.162	0.743	1.666	0.115	0.114	3	-
LSTM	1005	266.703	1929	0.687	0.682	17217	27.000
TCN	644.485	193.046	1371	0.371	0.368	5601	26.333

Hardware and measurement protocol are recorded in `results/hardware.json`: Intel Core i5-6300U (2 physical cores, 4 threads, 2.4 GHz nominal), 7.9 GB RAM, Windows 10, no GPU, PyTorch pinned to 2 threads.

These training times must be read with a large caveat, and it would be dishonest to present them as clean measurements. The runs were executed on a laptop that was simultaneously running the development environment, and free physical memory fell to roughly 466 MB of 8 GB during the evaluation. The distortion is directly visible in the numbers: the same LSTM configuration cost about 10.5 s per epoch during the tuning phase and about 88 s per epoch on square 5161 during the final run — an eightfold difference with no change in architecture, data or batch size. The `train_seconds_min` and `train_seconds_max` columns (LSTM 267 s to 1,929 s; TCN 193 s to 1,371 s) therefore measure machine load at least as much as they measure model cost.

Three comparisons are nonetheless robust, because they hold across the variation rather than depending on any single measurement.

SARIMA is faster by three orders of magnitude. It fits in 0.74 to 1.67 seconds, against hundreds to thousands of seconds for either network, and it has 3 parameters against 5,601 and 17,217. Given that it is also the least accurate model — the only one that loses to persistence — the trade-off here is unambiguous, but the direction is worth stating precisely: SARIMA is not a cheap approximation to the networks, it is a different and worse point on the accuracy axis that happens to be nearly free.

The TCN is cheaper than the LSTM despite being more accurate. Mean training 644 s against 1,005 s, and 5,601 parameters against 17,217 — roughly a third of the parameters. This is architectural rather than incidental: the TCN's convolutions over the 144-step window are computed in parallel across time positions, whereas the LSTM must step through all 144 positions sequentially, which does not parallelise across the two available cores. The usual expectation that a convolutional model trades accuracy for speed does not hold here; the TCN is better on both axes.

Inference is not a constraint for any model. The slowest, the LSTM, takes 0.68 ms per forecast step. A ten-minute-ahead forecast produced in under a millisecond leaves the operational decision latency-bound elsewhere, so inference cost should not influence model choice for this application — a point worth making because it is often assumed to.

One design consequence deserves recording. Bounding training by wall-clock time made the pipeline **not reproducible in the strict sense**, even with all seeds fixed: a differently loaded machine stops at a different epoch and returns different numbers. The budget was necessary to make the search affordable at all on this hardware, and epoch-matched comparison was used wherever an architectural question was at stake (Section 6.1). But it is a genuine methodological weakness rather than an implementation detail, and it is why the final predictions themselves are written to `results/predictions_square_*.csv` — those files, not the seed, are what pin the results in this report.

6.4 Failure analysis

Aggregate metrics hide *when* a model is wrong. Two diagnostics were computed: absolute error grouped by hour of day, and the worst contiguous six-hour window per model.

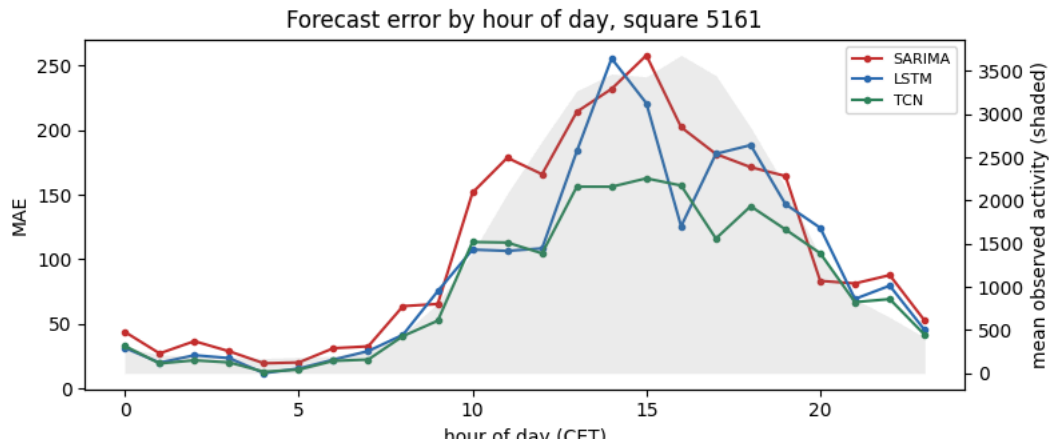


Figure 11. Square 5161: MAE by hour of day for the three models, with mean observed activity shaded behind. Error is a function of traffic level, and the models separate only at the peak.

Error is almost a function of the traffic level. Between 00:00 and 06:00 all three models sit between roughly 15 and 45 MAE and are indistinguishable from one another. Between 13:00 and 16:00, when mean activity is highest, SARIMA rises to about 258 and the LSTM to about 256, while the TCN stays near 163. The three models are effectively equivalent for two-thirds of the day.

This locates the TCN's entire advantage: it is not a uniformly better model, it is a better model **at the daily peak**, and because the peak dominates the mean absolute error, that is enough to produce a 15% overall improvement. It also explains why every model's $RMSE \div MAE$ exceeds 1.4 and why the ranking would change under a metric weighted towards overnight accuracy.

Worst six-hour windows. The same qualitative failure appears in all three areas, and in every case it is the midday-to-evening peak:

Area	Worst window	SARIMA	LSTM	TCN
5161	Sat 21 Dec, 13:10–19:00	304.0 (2.81×)	201.0 (2.16×)	148.2 (1.89×)
5059	Tue 17 Dec, 12:20–18:10	158.6 (1.78×)	173.7 (2.58×)	164.8 (2.39×)
5259	Mon 16 Dec, 12:20–18:10	221.7 (2.71×)	131.0 (1.95×)	118.6 (1.83×)

Errors in these windows run 1.8 to 2.8 times the weekly mean. No model escapes, which is itself the finding: the difficulty is a property of the peak, not of any one architecture.

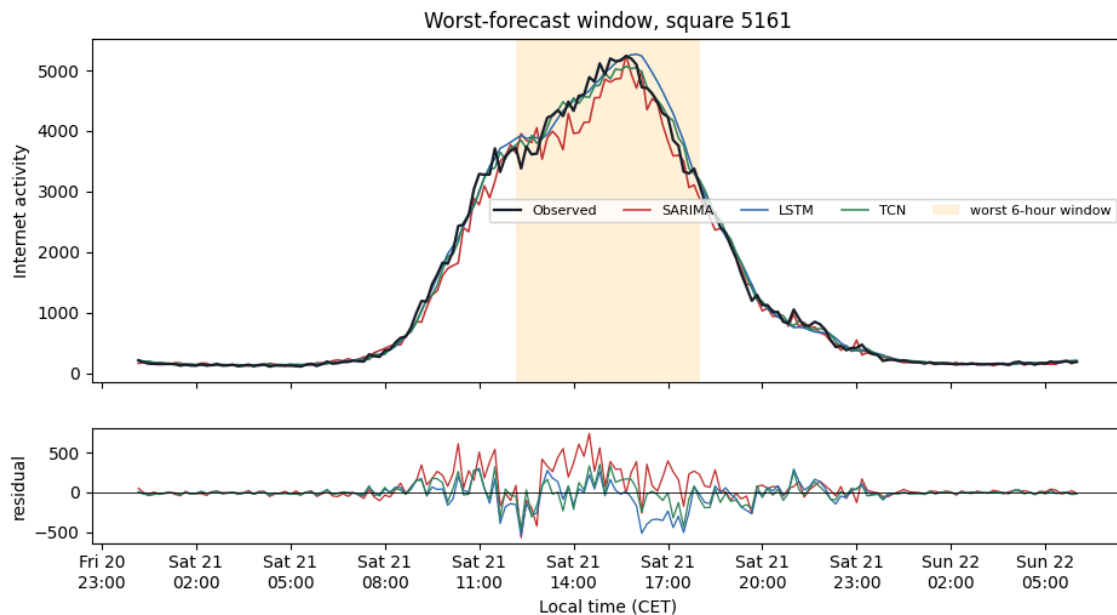


Figure 12. Square 5161, the worst window (shaded): Saturday 21 December, the highest-traffic day in the entire two-month record. Residuals below.

Square 5161's worst window is the most revealing, because it is not an arbitrary bad afternoon. **Saturday 21 December is the pre-Christmas shopping Saturday, and it carries the highest traffic of the whole two-month record** — roughly 5,200 activity units against an ordinary weekday peak near 3,500. Section 4.4 had flagged exactly this risk in advance: the training period, which ends on 8 December, contains no comparable pre-Christmas period, so the models are being asked to extrapolate to a level they have never observed.

The three models fail differently, and the differences are diagnostic rather than incidental:

- **SARIMA under-predicts systematically**, with a window bias of **-207** — it spends the afternoon below the observed series. Its residual panel also oscillates far more than the others. Both are consequences of its structure: the forecast is anchored to the same time on the previous day through the seasonal difference, and the previous day was an ordinary Friday. A model whose seasonal term says "yesterday" cannot represent a day that is unlike yesterday.
- **The LSTM over-predicts**, with a bias of **+112**, and the residual panel shows why: it tracks the ascent adequately but stays high as traffic falls after 17:00, overshooting the descent by up to about 500 units. It learned the shape of a typical peak and applied it to an atypical one.

- **The TCN is nearly unbiased at +15**, with the lowest window error (148.2). Its errors in this window are largely variance rather than a systematic misreading of the day.

That ordering — SARIMA biased low, LSTM biased high, TCN roughly centred — is consistent with the receptive-field finding of Section 6.1. The models that lean hardest on periodic structure (SARIMA explicitly, the LSTM through a 144-step window it must compress) are the ones misled when the period breaks. The TCN, whose selected configuration cannot even see a full day, has less periodic prior to be wrong about, and relies more on the immediately preceding observations — which on an anomalous day are the more reliable evidence.

A limitation this analysis exposes. Every model here is univariate: its only input is the area's own traffic history. A calendar feature marking the pre-Christmas period, or the shopping Saturday specifically, is exactly the information needed for the largest failure in the evaluation, and none of the models has access to it. This is a limitation of the experimental design rather than of the architectures, and Section 7 records it as the most promising extension.

6.5 Comparative summary

Bringing the three axes together:

	SARIMA	LSTM	TCN
Mean MAE ratio to persistence	1.112 (worse)	0.905	0.848
Spread of that ratio across areas	0.088	0.179	0.007
Areas where it beats persistence	0 of 3	2 of 3	3 of 3
Parameters	3	17,217	5,601
Mean training time	1.2 s	1,005 s	644 s
Inference per step	0.11 ms	0.68 ms	0.37 ms
Worst-window behaviour	biased low (−207)	biased high (+112)	near-unbiased (+15)

Which model should be preferred, and why. On this evidence the TCN, on three independent grounds rather than one: it is the most accurate on two of three areas and never worse than second; its margin over persistence is nearly invariant across areas with very different traffic character, which is what makes it deployable to cells it was not tuned on; and it achieves this with a third of the LSTM's parameters and roughly two-thirds of its training cost.

Where the comparison is closer than the headline suggests. The LSTM is genuinely the better model on square 5059, and its worst result — square 5161 — is the one run that was truncated by the compute budget. A fair statement is that the TCN is more *reliable* across areas, not that it is uniformly more accurate.

Relation to prior work. These results sit consistently with the literature reviewed in Section 2 once horizon is taken into account. Zhang and Patras [2] report large deep-learning gains that *grow with horizon* and shrink at short horizons; at a one-step horizon the gain observed here is 15%, not the 35–61% they report at long horizons, and the classical model does not merely narrow the gap but loses outright to persistence. The finding that a well-specified simple method is hard to beat matches Makridakis *et al.* [21] and the domain-specific observation of Wang *et al.* [6], where Holt-Winters outperformed their own LSTM. The TCN outperforming the LSTM is the direction Bai *et al.* [13] predict; the present result extends that claim to a seasonal real-valued forecasting task, which their benchmark suite did not include, though on a single dataset and with the single-seed limitation noted in Section 6.1.

7. Conclusion and Future Work

7.1 Findings

This study compared a seasonal ARIMA, an LSTM and a temporal convolutional network for one-step-ahead forecasting of Internet traffic in three areas of the Telecom Italia Milan grid, evaluated on the week of 16–22 December 2013 against persistence and seasonal-naïve baselines.

The choice of baseline decides what the results mean. Every model scored MASE far below 1 — that is, all comfortably beat "same time yesterday" — but seasonal naïve is a weak benchmark for this series, because lag-1 autocorrelation is 0.987 against 0.878 at lag 144. Measured against persistence instead, SARIMA is 7.6% to 16.4% *worse* on all three areas. A report that presented only the required metrics would have concluded that all three models succeeded.

The TCN was the most reliable model, and its consistency is the substantive result. It improved on persistence by 14.8%, 15.4% and 15.5% across the three areas — a spread of 0.007 — despite being tuned on one area only and applied unchanged to areas that differ fivefold in level and invert in weekend behaviour. The LSTM's mean advantage was similar (0.905 against 0.848) but its spread was 25 times larger, ranging from a 17.5% improvement to merely tying persistence. Mean accuracy alone would have made these two models look comparable.

The linear model lost on predictive accuracy while winning decisively on cost. SARIMA fits in about one second with three parameters against hundreds of seconds and thousands of parameters, and its inference is six times faster than the LSTM's. It is not a cheap approximation to the networks; it is a worse forecaster that is nearly free.

Receptive field did not matter, and the reason is instructive. A controlled experiment with matched epochs found no measurable effect on accuracy across receptive fields from 61 to 509 steps; the non-monotonicity of the results bounded run-to-run variation at roughly 5 MAE units, seven times the difference between the extremes. A TCN that provably cannot see "same time yesterday" matched one covering three and a half days. The strong daily seasonality established in the exploratory analysis is real but largely redundant at a one-step horizon, where recent observations already imply it. It is decisive for SARIMA, which must represent the cycle explicitly, and nearly irrelevant to a network conditioned on recent history.

Errors are concentrated where traffic is highest. All three models were equivalent overnight and separated only at the daily peak, where the TCN's advantage was earned entirely. The worst six-hour window in every area was the midday-to-evening peak, at 1.8 to 2.8 times the weekly mean error, and on the busiest area it was Saturday 21 December — the pre-Christmas shopping Saturday and the highest-traffic day in the entire record, a level the training period never contained.

Returning to the research question: the models differ substantially, the convolutional model is preferable to the recurrent one on accuracy, cost *and* cross-area stability, and the classical linear model is not competitive at this horizon. Performance does vary across areas — but for the TCN it varies far less than the areas themselves do, which is the more useful conclusion for an operator who cannot tune a model per cell.

7.2 Limitations

Stated in order of how much they constrain the conclusions.

1. **Single seed per configuration.** Neural runs were trained once. The receptive-field study's own non-monotonicity implies run-to-run variation of about 5 MAE units, which is comparable to some of the differences discussed. The cross-area *consistency* results are robust to this, since a spread of 0.007 across three areas is not plausibly luck, but individual pairwise comparisons — notably LSTM against TCN on square 5059, a gap of 1.6 MAE — are not resolved by this evidence.
2. **Wall-clock budgets make the pipeline non-reproducible in the strict sense.** Training was bounded by time as well as epochs, so a differently loaded machine stops at a different epoch. One final run, the LSTM on square 5161, was truncated — and it is the run behind the LSTM's weakest result. Predictions are archived to `results/predictions_square_*.csv` because the seed alone does not pin the numbers.

3. **Timing measurements are contaminated by machine load.** The same LSTM configuration cost 10.5 s per epoch during tuning and 88 s per epoch during the final run. Only the order-of-magnitude comparisons and the relative LSTM/TCN ordering should be trusted.
4. **Three areas, all high-traffic.** Squares 5161, 5059 and 5259 are the three busiest of 10,000 and are geographically adjacent in the historic centre. Section 4.1 showed the bottom 50% of areas carry 11.6% of traffic; nothing here establishes how these models behave on a sparse suburban cell, where zero-valued intervals are common and persistence may be even harder to beat.
5. **SARIMA was restricted to $P = Q = 0$.** Seasonal AR and MA terms were excluded for the computational reasons given in Section 5.4. The evidence that seasonal differencing alone achieves stationarity makes this defensible, but the model class was not fully explored, and SARIMA's poor showing should be read with that in mind.
6. **Univariate inputs only.** No calendar, weather, spatial-neighbour or cross-channel information was used — and the largest failure in the evaluation is precisely a calendar effect.
7. **Symmetric error metrics.** As Bega *et al.* [8] argue, over- and under-provisioning carry asymmetric operational costs, so MAE and RMSE do not rank models the way an operator's cost function would. SARIMA's systematic under-prediction at peaks would be penalised far more heavily under an asymmetric cost than these tables suggest.

7.3 Future work

Ordered by expected return relative to effort, and each one motivated by a specific finding above rather than by generic ambition.

1. **Add calendar features.** The single highest-value extension, because it targets the largest observed failure directly. Day-of-week, hour-of-day and an Italian public-holiday and pre-Christmas indicator would give the models the information they demonstrably lacked on 21 December. Cheap to implement and testable within the existing harness.
2. **Repeat the key comparisons across seeds.** Five seeds per configuration would convert the LSTM-versus-TCN comparison from suggestive to statistically supported, and would replace the crude noise bound of Section 6.1 with a measured variance. This is the cheapest way to strengthen the existing claims and, given the timing distortions documented above, should be run on an unloaded machine.
3. **Extend to a horizon sweep.** The most interesting implication of this work is that the daily cycle is largely redundant at one step. Evaluating at 1, 6, 36 and 144 steps ahead would test the prediction that follows: receptive field and seasonal structure should become progressively more important as the horizon grows, and SARIMA's deficit relative to persistence should shrink and eventually reverse. This would also connect the results directly to Zhang and Patras [2], whose gains are reported at long horizons.
4. **Sample areas across the traffic distribution.** Repeating the evaluation on cells drawn from the median and the bottom decile would test whether the TCN's cross-area stability extends beyond the busy centre, which is the claim most likely to be over-generalised from the present evidence.
5. **Exploit spatial structure.** The literature's strongest results use spatio-temporal models [4], [15], and Section 4.1 showed traffic decays smoothly from a single core, implying that neighbouring cells carry usable signal. This is the largest potential gain but also the largest engineering step, and it would require re-ingesting data the current pipeline deliberately discards.
6. **Train against an asymmetric cost.** Following DeepCog [8], replacing the Huber loss with a cost that penalises under-provisioning more heavily would align the objective with the operational use case that motivates the problem.

8. Use of AI Assistance

Note to the author: verify this section against your own recollection and edit it before submitting. A disclosure is only useful if it is accurate.

An AI coding assistant (Cursor, using a large language model) was used substantially during this project. The disclosure below is specific rather than general, because a vague acknowledgement would not serve its purpose.

Where AI assistance was used.

- *Code.* The assistant produced the initial implementation of most modules in `src/`, including the streaming ingestion pipeline, the memory benchmark harness, the model wrappers, the metric definitions, and the plotting code. It also wrote the unit tests in `tests/`.
- *Debugging.* The Harvard Dataverse access problem (HTTP 403 from a missing `User-Agent`, then HTTP 400 from the download guestbook) and the parallel range-request downloader were diagnosed and implemented with AI assistance.
- *Literature search.* Candidate references were located with AI assistance and each bibliographic record was then checked against a publisher, repository or Crossref record; the verification status of every reference, including the fields that could *not* be confirmed, is recorded in `report/related_work_notes.md`.
- *Report drafting.* The prose of this report was drafted with AI assistance and then revised.

Where it was not, and what was checked independently.

- Every number quoted in this report is generated by the code in this repository and is traceable to a file in `results/`. No result was written from memory or estimated.
- Several AI-produced errors were found and corrected during the work, and they are worth listing because they indicate the kind of checking that was done: `statsmodels` silently fitting a constant under `trend=None` (Section 5.4); an inverse transform that could return negative traffic volumes (Section 5.2); a cache that could serve spatial aggregates computed from a partially ingested store; and a draft claim that the receptive-field study used multiple random seeds when the code in fact ran each configuration once (Section 6.1).
- The MASE denominator used here departs from the published definition, and that departure is stated in Section 5.2 rather than left implicit.
- The confound in the first TCN tuning study — receptive field varying together with the number of epochs actually trained — was identified during analysis and addressed by designing the controlled study in Section 6.1 rather than by reporting the original ranking.

Responsibility. The design of the experiments, the interpretation of the results, and the conclusions drawn are the author's own, and the author is able to explain and justify every component of the submitted work.

9. References

- [1] G. Barlacchi, M. De Nadai, R. Larcher, A. Casella, C. Chitic, G. Torrisi, F. Antonelli, A. Vespignani, A. Pentland, and B. Lepri, "A multi-source dataset of urban life in the city of Milan and the Province of Trentino," *Scientific Data*, vol. 2, art. no. 150055, Oct. 2015, doi: 10.1038/sdata.2015.55.
- [2] C. Zhang and P. Patras, "Long-term mobile traffic forecasting using deep spatio-temporal neural networks," in *Proc. 19th ACM Int. Symp. Mobile Ad Hoc Networking and Computing (MobiHoc '18)*, Los Angeles, CA, USA, Jun. 2018, pp. 231–240, doi: 10.1145/3209582.3209606.

- [3] C. Zhang, H. Zhang, D. Yuan, and M. Zhang, "Citywide cellular traffic prediction based on densely connected convolutional neural networks," *IEEE Communications Letters*, vol. 22, no. 8, pp. 1656–1659, Aug. 2018, doi: 10.1109/LCOMM.2018.2841832.
- [4] C. Zhang, H. Zhang, J. Qiao, D. Yuan, and M. Zhang, "Deep transfer learning for intelligent cellular traffic prediction based on cross-domain big data," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 6, pp. 1389–1401, Jun. 2019, doi: 10.1109/JSAC.2019.2904363.
- [5] C.-W. Huang, C.-T. Chiang, and Q. Li, "A study of deep learning networks on mobile traffic forecasting," in *Proc. IEEE 28th Annu. Int. Symp. Personal, Indoor, and Mobile Radio Communications (PIMRC)*, Montreal, QC, Canada, Oct. 2017, pp. 1–6, doi: 10.1109/PIMRC.2017.8292737.
- [6] X. Wang, Z. Zhou, F. Xiao, K. Xing, Z. Yang, Y. Liu, and C. Peng, "Spatio-temporal analysis and prediction of cellular traffic in metropolis," *IEEE Transactions on Mobile Computing*, vol. 18, no. 9, pp. 2190–2202, Sep. 2019, doi: 10.1109/TMC.2018.2870135.
- [7] H. D. Trinh, L. Giupponi, and P. Dini, "Mobile traffic prediction from raw data using LSTM networks," in *Proc. IEEE 29th Annu. Int. Symp. Personal, Indoor and Mobile Radio Communications (PIMRC)*, Bologna, Italy, Sep. 2018, pp. 1827–1832, doi: 10.1109/PIMRC.2018.8581000.
- [8] D. Bega, M. Gramaglia, M. Fiore, A. Banchs, and X. Costa-Pérez, "DeepCog: Cognitive network management in sliced 5G networks with deep learning," in *Proc. IEEE INFOCOM 2019 — IEEE Conf. Computer Communications*, Paris, France, Apr. 2019, pp. 280–288, doi: 10.1109/INFOCOM.2019.8737488.
- [9] C. Zhang, P. Patras, and H. Haddadi, "Deep learning in mobile and wireless networking: A survey," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2224–2287, 3rd Quart. 2019, doi: 10.1109/COMST.2019.2904897.
- [10] Y. Shu, M. Yu, J. Liu, and O. W. W. Yang, "Wireless traffic modeling and prediction using seasonal ARIMA models," in *Proc. IEEE Int. Conf. Communications (ICC)*, May 2003, vol. 3, pp. 1675–1679, doi: 10.1109/ICC.2003.1203886.
- [11] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: <https://otexts.com/fpp3/>
- [12] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997, doi: 10.1162/neco.1997.9.8.1735.
- [13] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," arXiv:1803.01271 [cs.LG], Mar. 2018, doi: 10.48550/arXiv.1803.01271. (Technical report; not a peer-reviewed conference paper.)
- [14] A. van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu, "WaveNet: A generative model for raw audio," arXiv:1609.03499 [cs.SD], Sep. 2016, doi: 10.48550/arXiv.1609.03499.
- [15] X. Shi, Z. Chen, H. Wang, D.-Y. Yeung, W.-K. Wong, and W.-C. Woo, "Convolutional LSTM network: A machine learning approach for precipitation nowcasting," in *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, Montreal, QC, Canada, Dec. 2015. [Online]. Available: <https://arxiv.org/abs/1506.04214>
- [16] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *International Journal of Forecasting*, vol. 22, no. 4, pp. 679–688, Oct.–Dec. 2006, doi: 10.1016/j.ijforecast.2006.03.001.
- [17] R. B. Cleveland, W. S. Cleveland, J. E. McRae, and I. Terpenning, "STL: A seasonal-trend decomposition procedure based on loess," *Journal of Official Statistics*, vol. 6, no. 1, pp. 3–73, 1990.
- [18] X. Wang, K. Smith, and R. Hyndman, "Characteristic-based clustering for time series data," *Data Mining and Knowledge Discovery*, vol. 13, no. 3, pp. 335–364, Nov. 2006, doi: 10.1007/s10618-005-0039-x.
- [19] D. A. Dickey and W. A. Fuller, "Distribution of the estimators for autoregressive time series with a unit root," *Journal of the American Statistical Association*, vol. 74, no. 366, pp. 427–431, Jun. 1979, doi: 10.1080/01621459.1979.10482531.
- [20] D. Kwiatkowski, P. C. B. Phillips, P. Schmidt, and Y. Shin, "Testing the null hypothesis of stationarity against the alternative of a unit root: How sure are we that economic time series have a unit root?," *Journal of Econometrics*, vol. 54, no. 1–3, pp. 159–178, Oct.–Dec. 1992, doi: 10.1016/0304-4076(92)90104-Y.
- [21] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "Statistical and machine learning forecasting methods: Concerns and ways forward," *PLoS ONE*, vol. 13, no. 3, art. no. e0194889, Mar. 2018, doi: 10.1371/journal.pone.0194889.
- [22] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M4 Competition: 100,000 time series and 61 forecasting methods," *International Journal of Forecasting*, vol. 36, no. 1, pp. 54–74, Jan.–Mar. 2020, doi: 10.1016/j.ijforecast.2019.04.014.
- [23] GSMA Intelligence, "Going green: Benchmarking the energy efficiency of mobile networks (second edition)," GSMA Intelligence, London, U.K., Feb. 2023. (Industry report; not peer-reviewed.)
- [24] L. M. P. Larsen, H. L. Christiansen, S. Ruepp, and M. S. Berger, "Toward greener 5G and beyond radio access networks—A survey," *IEEE Open Journal of the Communications Society*, vol. 4, pp. 768–797, 2023, doi: 10.1109/OJCOMS.2023.3257889.
- [25] J. Wu, Y. Zhang, M. Zukerman, and E. K.-N. Yung, "Energy-efficient base-stations sleep-mode techniques in green cellular networks: A survey," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 2, pp. 803–826, 2nd Quart. 2015, doi: 10.1109/COMST.2015.2403395.
- [26] P. J. Huber, "Robust estimation of a location parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, Mar. 1964, doi: 10.1214/aoms/1177703732.
- [27] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, May 2015. [Online]. Available: <https://arxiv.org/abs/1412.6980>

Software

- [28] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, Sep. 2020, doi: 10.1038/s41586-020-2649-2.
- [29] S. Seabold and J. Perktold, "statsmodels: Econometric and statistical modeling with Python," in *Proc. 9th Python in Science Conf. (SciPy)*, Austin, TX, USA, 2010, pp. 92–96, doi: 10.25080/Majora-92bf1922-011.
- [30] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, Vancouver, BC, Canada, Dec. 2019. [Online]. Available: <https://arxiv.org/abs/1912.01703>

Project artefacts

- [31] Source code repository: *[insert GitHub URL before submission]*
- [32] Video presentation: *[insert video URL before submission]*